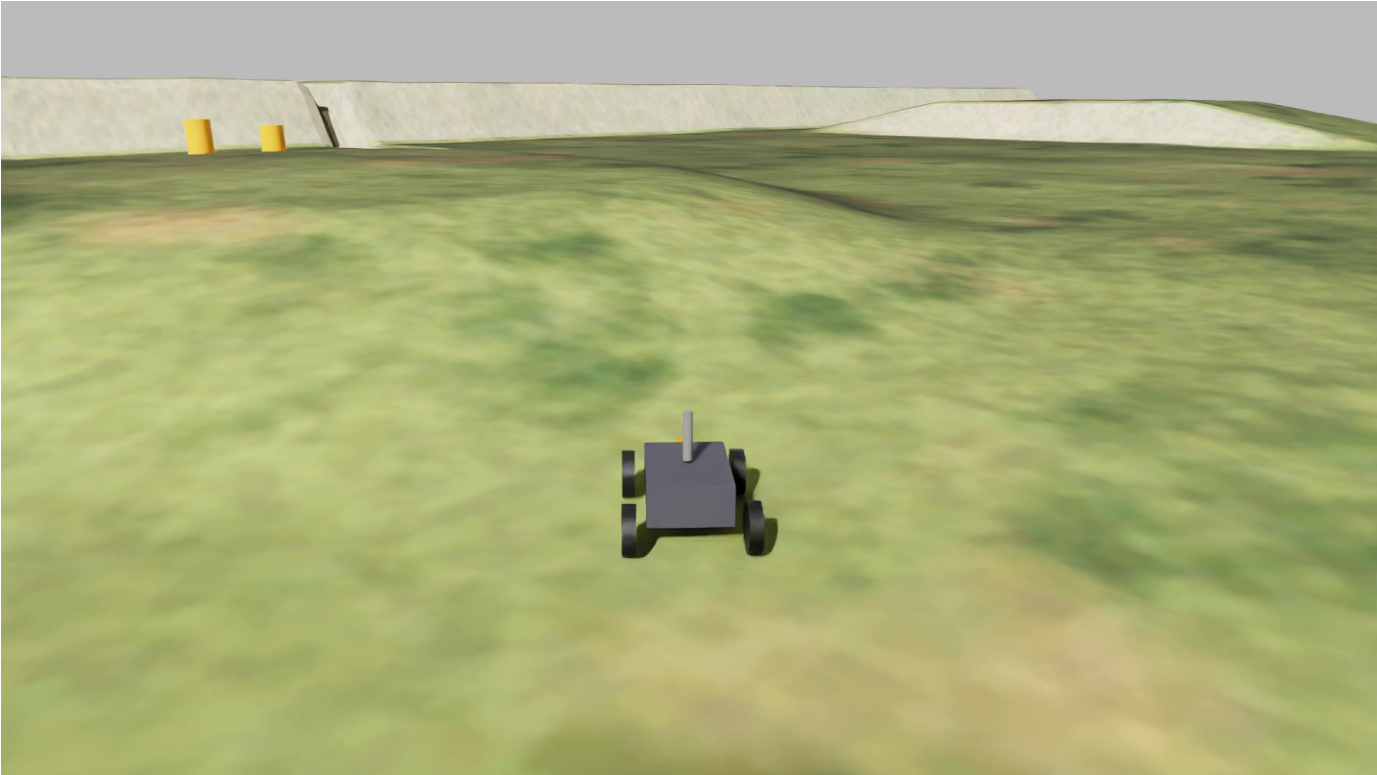


QSLAM

Vision-based, GPS-denied autonomous navigation for an unmanned ground vehicle in unstructured outdoor terrain

Detailed system description — every phase, algorithm, ROS 2 node, map format and design decision, with the measured results, followed by a plain-language explanation for readers new to robotics.



The rover approaching the 3.2 m earth berm and its tunnel bore in the Isaac Sim 6.0.1 proving ground — frame from a recorded GPS-free mission.

Problem statement	SIH26126 — “Vision Based Autonomous Navigation for Unmanned Ground Vehicle for Outdoor environment”
Theme • category	Smart Automation • Software
Team	QSLAM (Team ID 14)
System	ROS 2 Jazzy • Nav2 • robot_localization EKF • Isaac Sim 6.0.1 • MuJoCo • Gazebo Harmonic
GNSS / GPS used	None, at any stage — no receiver, no NavSatFix, no navsat_transform
Document	Version 1.0 • 28 September 2026 • generated by src/sih_isaac/scripts/make_technical_pdf.py

How to read this document
Part A — Technical description (Sections 1–17): architecture, each development and mission phase, and how every algorithm works — with equations, flowcharts, node graphs and the EKF estimation graphs from recorded runs. Part B — QSLAM for everyone (last five pages): what the system does, whether it uses GPS, whether it is online or offline, and what has been proven — no background needed.

Contents

1 Problem, requirements and design principles	3
2 The system at a glance	3
3 Development phases and their architecture	4
4 Software architecture	7
5 Georeferencing without GPS: UTM and the surveyed datum	9
6 The digital elevation model (DEM)	10
7 GeoTIFF – the single source of truth	12
8 Traversability and the prior occupancy map	13
9 Costmaps	13
10 Global planner – A* (Nav2 Smac Planner 2D)	14
11 Local planner – Model Predictive Path Integral control (MPPI)	15
12 Behaviour tree, supervision and recovery	18
13 Localization – the Extended Kalman Filter	19
14 Visual SLAM	21
15 QSLAM innovations – designed modules	23
16 Simulation platform, vehicle and results	24
17 Technology stack, limitations and roadmap	25
Part B – QSLAM explained for everyone	26

Table 1. Status labels used throughout Part A.

Status label	Meaning in this document
IMPLEMENTED · Isaac testbed	Runs in this workspace in the Isaac Sim 6.0.1 integration testbed (src/sih_isaac) and was measured.
IMPLEMENTED · MuJoCo prototype	Runs in the MuJoCo end-to-end prototype (src/sih_demo).
IMPLEMENTED · Gazebo Phase 1	Gazebo Harmonic world, robot, sensors and bridge (src/sih_sim, src/sih_description).
EXPERIMENTAL / NOT WIRED	Installed or wired but not part of the measured configuration (cuVSLAM, slam_toolbox).
QSLAM DESIGN · planned	Part of the registered QSLAM design; specified here, not yet in the repository.

Three answers up front.

- **Does QSLAM use GPS?** No. There is no GNSS receiver in the system at all. Position comes from a surveyed start point, the vehicle's own motion sensing (visual-inertial odometry + IMU heading) and a preloaded georeferenced map.
- **Online or offline?** Fully offline with respect to networks: no internet, cloud or radio link is needed. Maps are prepared once before the mission; during the mission everything runs on the vehicle's own computer in real time.
- **What is proven?** In the Isaac Sim testbed the rover drove to UTM goals through a tunnel and a corridor of unsurveyed obstacles and walking people, with 0.98–1.42 m localization error at the end of 123–295 m missions without GPS (Section 16).

1 Problem, requirements and design principles

SIH26126 asks for a software module that drives an unmanned ground vehicle (UGV) to a goal across unstructured outdoor terrain using vision-based navigation. The operator specifies the destination as a **UTM coordinate** (easting/northing), the terrain contains slopes, ditches, vegetation and man-made obstacles, and the vehicle must not rely on satellite positioning. GNSS is exactly what disappears where UGVs are most needed: in tunnels, under canopy, beside tall structures, and wherever signals are jammed or spoofed. QSLAM treats that as a design rule rather than a failure mode: **there is no GNSS receiver anywhere in the software**, so the system cannot silently fall back on one.

Table 2. Requirements of the problem statement and how QSLAM meets them.

Requirement	Design decision in QSLAM	Where
Goal given in UTM, no GNSS	Surveyed site datum converted to a UTM anchor offline (pyproj); at run time the goal is anchor-subtracted — pure arithmetic, nothing measured	\$5, utm_goal.py
Unstructured terrain (slopes, ditch, berm, tunnel)	Prior digital elevation model stored as a GeoTIFF; slope > 15° marked non-traversable; global route planned on it	\$6–8
Unknown and moving obstacles (drums, logs, people)	Live lidar obstacle layer in rolling costmaps; MPPI local controller re-optimises 15 times per second	\$9, \$11
Reliable pose without GPS on a skid-steer	EKF fusing VIO-grade body velocity with absolute IMU/AHRS heading — chosen after wheel odometry failed in measurement	\$13
Vision-based localization	Visual SLAM front-end (prototype), cuVSLAM stereo VIO (installed), map anchoring to satellite/DEM (QSLAM design)	\$14, \$15
Reproducible validation	Isaac Sim world generated from the same GeoTIFF the planner reads; ground truth logged only for scoring	\$7, \$16
Transferable to hardware	Standard ROS 2 Jazzy + Nav2 interfaces; Jetson AGX Orin as target computer	\$4, \$17

1.1 Five engineering rules

A previous attempt at this problem lost time to wiring mistakes that looked like algorithm failures. The five rules below are enforced in every launch and configuration file of this workspace:

- **No GNSS in the graph** — no `NavSatFix` subscriber, no `navsat_transform_node`, no `gps_*` topic, parameter or script.
- **REP-103 camera frames** — optical frame (z forward, x right, y down) separate from the body frame (x forward, y left, z up), joined by a static transform; the `frame_id` actually published is verified, not assumed.
- **Exactly one publisher per TF edge** — every non-owner node has TF publishing disabled (Section 4.3).
- **Never set odom_frame_id on the rtabmap node** — it would read odometry from TF and silently ignore the visual-odometry topic.
- **Build products are never committed** — `build/`, `install/`, `log/`, generated worlds and model weights are git-ignored.

2 The system at a glance

QSLAM splits cleanly into what happens **once, before the mission** (offline preparation of maps and the georeference) and what happens **continuously during the mission** (onboard sensing, estimation, planning and control). Figure 1 shows the split and the three things the system deliberately does not use.

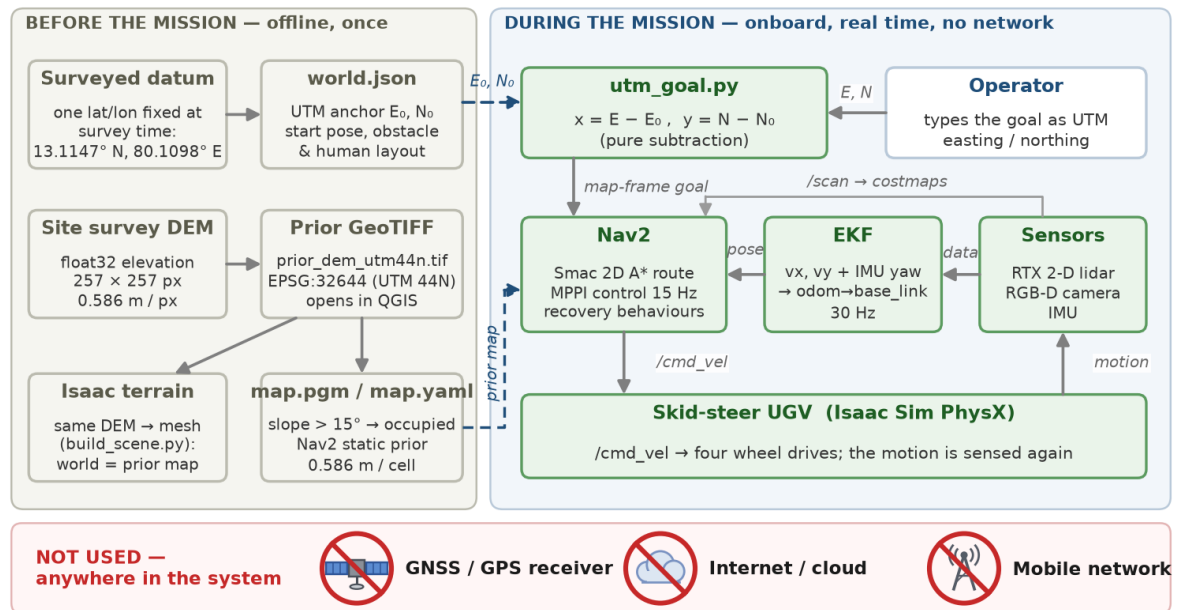


Figure 1. System context. Left: offline preparation, run once. Right: the onboard real-time loop. No GNSS receiver, internet, cloud service or mobile network is used at any point.

Table 3. QSLAM system summary.

Aspect	QSLAM (this workspace)
Vehicle	4-wheel skid-steer UGV, 60 kg, wheelbase 0.80 m, track 0.70 m, wheel radius 0.15 m, chassis 0.70 × 0.50 × 0.25 m
Sensors	RGB-D camera (ZED-class, 640 × 360 in Isaac), mast-top RTX lidar at 0.45 m (2-D scan), IMU; wheel joint states
Localization	robot_localization EKF (30 Hz, 2-D): body velocity + absolute heading; surveyed start pose. cuVSLAM installed as the visual odometry source
Maps	Prior DEM as GeoTIFF (EPSG:32644, 257 × 257 px, 0.586 m/px) → slope-derived occupancy map; live lidar costmaps
Global planner	Nav2 Smac Planner 2D (A* on the global costmap), replanned at 1 Hz
Local planner	Nav2 MPPI controller, 1800 samples × 56 steps (3.75 s), 15 Hz
Supervision	Nav2 behaviour tree with recovery (clear costmaps, spin, wait, back up)
Middleware / tools	ROS 2 Jazzy, Nav2, Isaac Sim 6.0.1 (PhysX, RTX sensors, OmniGraph ROS 2 bridge), RViz 2, rasterio, pyproj, OpenCV, MuJoCo, Gazebo Harmonic
Positioning infrastructure	None: no GNSS, no beacons, no network. Only a surveyed site datum and preloaded map files

3 Development phases and their architecture

QSLAM was built in gated phases: each phase had an exit checklist and its numbers were measured before the next began. Figure 2 shows the sequence and the status of each phase; the table summarises what each phase contains.

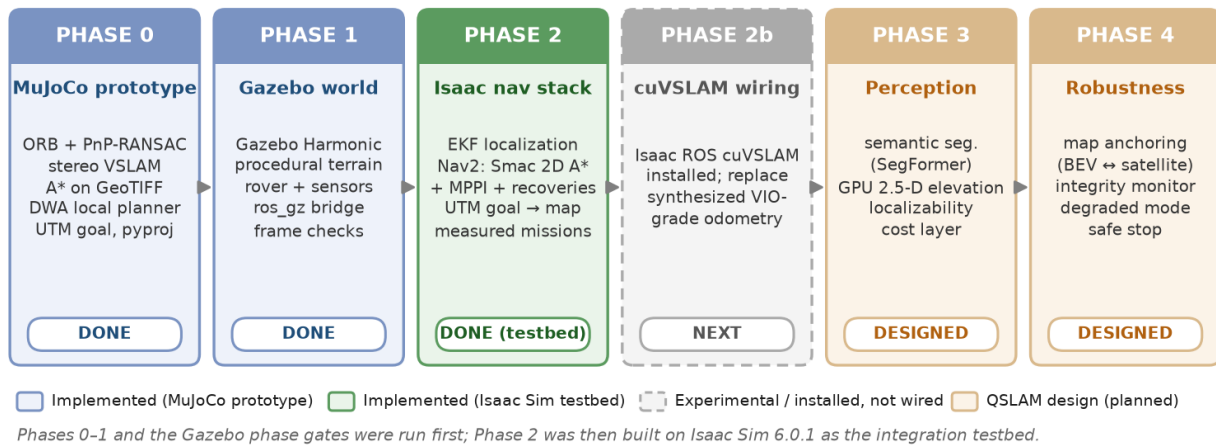


Figure 2. Development phases. Green and blue phases run in this workspace; grey is installed but not yet wired; orange is the registered QSLAM design still to be built.

Table 4. Development phases: content, architecture and status.

Phase	Architecture and algorithms	Status and evidence
0 · MuJoCo end-to-end prototype (src/sih_demo)	ZED 2i RGB-D rendering → ORB + PnP-RANSAC visual odometry (§14.2); depth → log-odds local costmap; GeoTIFF prior → slope-weighted A* (§10.4); DWA local planner (§11.2); UTM↔map via pyproj	IMPLEMENTED · MuJoCo prototype Goal reached, 0 contacts with 21 unsurveyed obstacles, VSLAM drift 0.98–4.3 % of path across runs, UTM round trip 0.000 mm
1 · Gazebo Harmonic world (src/sih_sim, sih_description)	Seeded analytic terrain rasterised to a 513 × 513 heightmap; xacro rover (single source for robot_state_publisher and spawn); RGB-D 1280 × 720 @ 30 Hz, IMU 200 Hz, joint states 50 Hz; ros_gz_bridge with verified topic names and frame_id overrides	IMPLEMENTED · Gazebo Phase 1 Sensors verified against REP-103; point-cloud frame found to be body-convention and bridged as camera_link
2 · Isaac Sim navigation stack (src/sih_isaac)	GeoTIFF as single source of truth → USD terrain + Nav2 map; EKF (VIO-grade velocity + absolute heading); Nav2 Smac 2D A* + MPPI + behaviour tree; UTM goal client; telemetry and recording	IMPLEMENTED · Isaac testbed Goal reached through the tunnel in every recorded run; 0.98–1.42 m localization error
2b · Visual odometry in the loop	Isaac ROS cuVSLAM (stereo/RGB-D VIO) replacing the synthesised velocity input of the EKF; RTAB-Map / slam_toolbox for map→odom	EXPERIMENTAL / NOT WIRED cuVSLAM 15.0.0 installed and configured; slam_toolbox wired behind slam:=true. Target: ≤ 2 % arrival error of distance
3 · Perception	Semantic segmentation (SegFormer, RELLIS-3D + RUGD, TensorRT) + GPU 2.5-D elevation mapping → traversability and localizability cost layers	QSLAM DESIGN · planned sih_perception package scaffolded. Target: ≥ 15 FPS on Jetson AGX Orin
4 · Robustness	Map-anchored re-localization (BEV ↔ satellite/DEM), localization integrity monitor, degraded-mode supervisor, Collision Monitor	QSLAM DESIGN · planned Target: ≤ 5 m / 2° start alignment, ≥ 90 % mission success

The prototype and the testbed share one world definition: `sih_sim/terrain.py` is the analytic terrain used by the Gazebo generator, the MuJoCo demo and the Isaac world, so results across phases describe the same site (seed 42). The remainder of Part A describes Phase 2 in depth, and Phases 0, 2b, 3 and 4 where their algorithms differ. After Phase 4 comes the hardware port (ZED X, AHRS, Jetson AGX Orin) and field trials, target ≤ 100 ms sensing-to-command latency; the targets are those of the QSLAM idea submission.

3.1 Mission phases – what happens when the system runs

Figure 3 follows one mission from survey to arrival. **M0 Preparation** (offline): world.py writes the GeoTIFF, the occupancy map and world.json; build_scene.py turns the same DEM into the Isaac USD stage. **M1 Bring-up**: the simulator starts publishing /clock and sensors; nav.launch.py starts the EKF seeded at the surveyed pose, the map server and Nav2 under a lifecycle manager. **M2 Goal**: the operator types a UTM easting/northing, which is anchor-subtracted, bounds-checked and sent as a NavigateToPose action. **M3 Planning**: Smac 2D computes the route (and recomputes it every second). **M4 Autonomy loop**: sense → localize → update costmaps → MPPI command, with recoveries when progress stalls. **M5 Arrival**: success inside 0.7 m, reported in map metres and UTM.

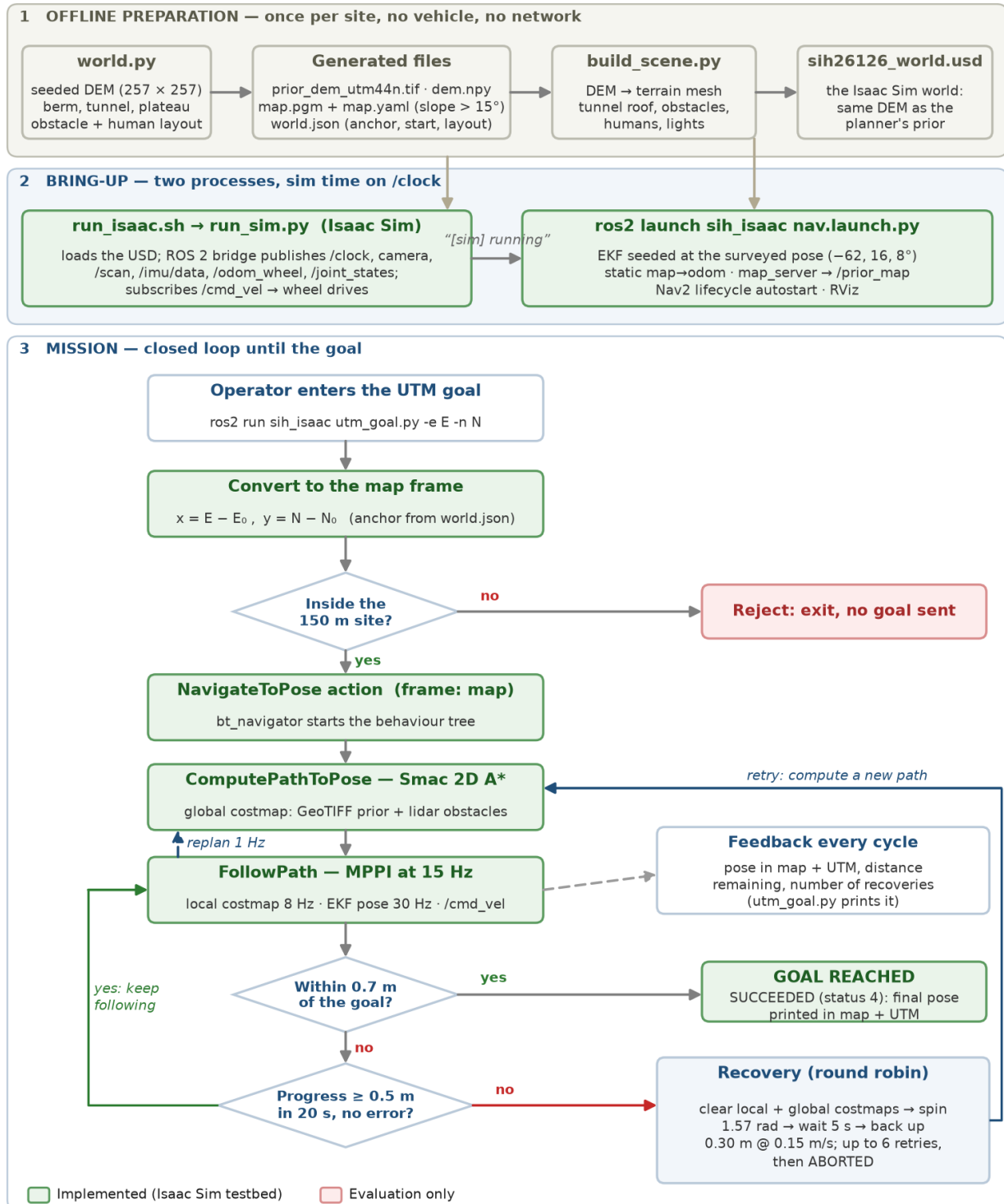


Figure 3. Mission flowchart from offline preparation to arrival, including the decision points and the recovery loop.

4 Software architecture

4.1 Layered view

The software is organised in five layers (Figure 4). Each layer only consumes the outputs of the layers below it through ROS 2 topics, actions and the TF tree, which is what makes the simulator replaceable by a real vehicle: only the platform layer changes.

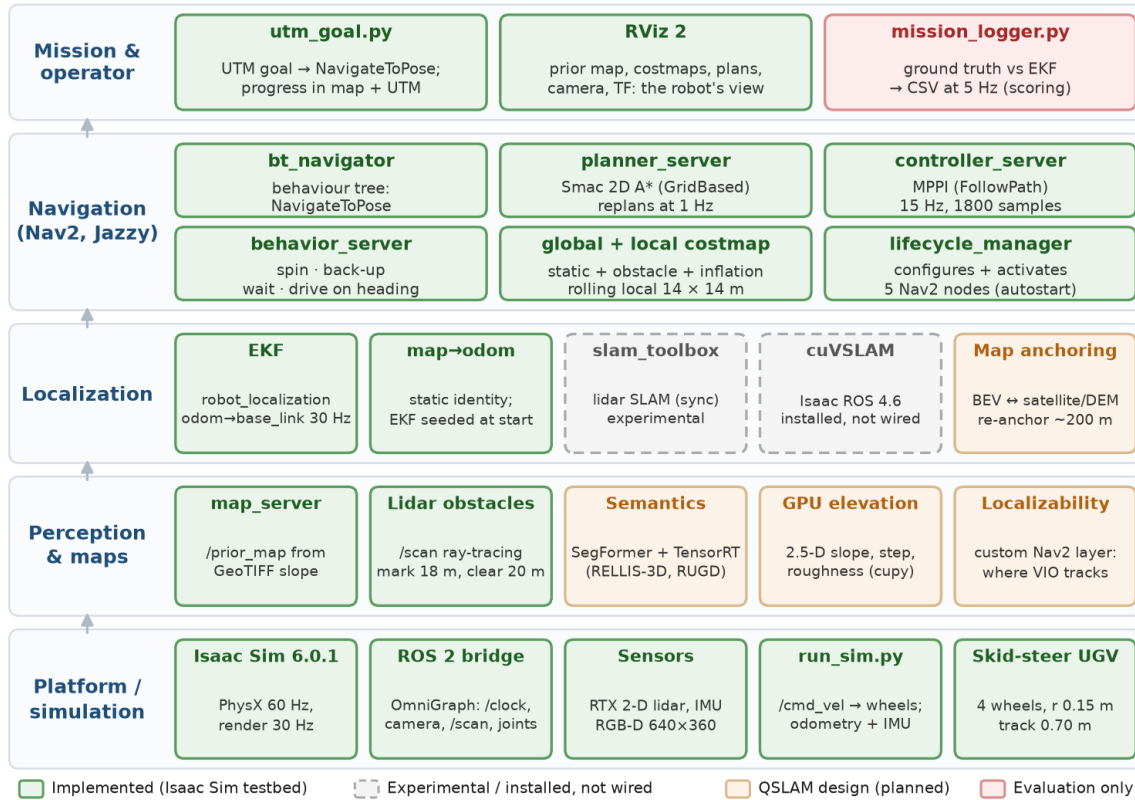


Figure 4. Layered architecture with the implementation status of every component.

4.2 ROS 2 computation graph

Figure 5 is the node/topic graph of the running testbed. Isaac Sim hosts one in-process ROS node (sih_isaac_sim) plus OmniGraph bridge nodes; everything else runs from nav.launch.py. Note that **/ground_truth** is consumed only by the evaluation logger – no navigation node subscribes to it.

Table 5. Topics and actions of the Isaac testbed.

Topic	Type	Publisher → subscribers	Rate / frame
/clock	rosgraph_msgs/Clock	Isaac (OmniGraph) → every node (use_sim_time)	every sim step
/scan	sensor_msgs/LaserScan	Isaac RTX lidar → global & local costmaps	render rate · lidar_link
/camera/image, /camera/depth_image, /camera/camera_info	sensor_msgs/Image, CameraInfo	Isaac → RViz (cuVSLAM input in Phase 2b)	30 Hz · camera_optical_frame
/imu/data	sensor_msgs/Imu	sih_isaac_sim → EKF	30 Hz · imu_link
/odom_wheel	nav_msgs/Odometry	sih_isaac_sim → EKF (twist only; no TF)	30 Hz · odom→base_link
/joint_states	sensor_msgs/JointState	Isaac → robot_state_publisher	render rate
/odometry/filtered	nav_msgs/Odometry	EKF → controller, BT navigator, logger	30 Hz · odom
/prior_map	nav_msgs/OccupancyGrid	map_server → global costmap static layer	latched (transient local) · map
/plan, /local_plan, /trajectories	Path, MarkerArray	planner / controller → RViz	1 Hz / 15 Hz
/cmd_vel	geometry_msgs/Twist	controller_server or behavior_server → sih_isaac_sim	15 Hz (0.5 s watchdog)

Topic	Type	Publisher → subscribers	Rate / frame
/ground_truth	nav_msgs/Odometry	sih_isaac_sim → mission_logger only	30 Hz · map (evaluation)
navigate_to_pose	nav2_msgs/action/NavigateToPose	utm_goal.py (client) → bt_navigator (server)	per mission

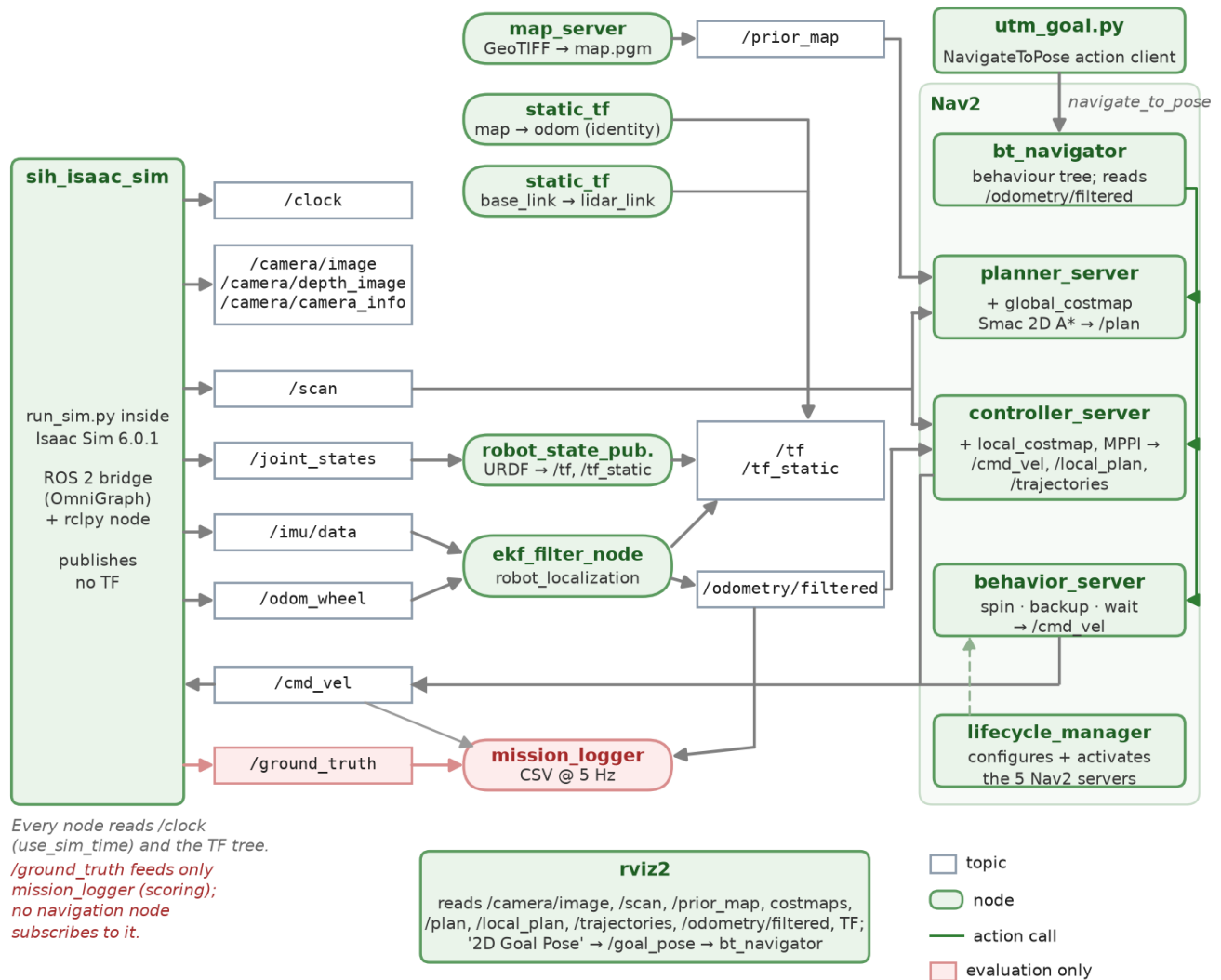


Figure 5. ROS 2 nodes (boxes) and topics/actions (labels) of the Isaac testbed.

4.3 Transform tree and ownership

In ROS the coordinate frames form a tree that every node queries. Two writers on one edge corrupt it silently, so each edge has exactly one owner (Figure 6). Isaac Sim publishes no TF. The static identity on map→odom is legitimate because the EKF is seeded with the surveyed deployment pose — odom coincides with map at $t = 0$ and only drift separates them afterwards; the map-anchoring module of the QSLAM design is exactly the component that will own this edge and remove that drift.

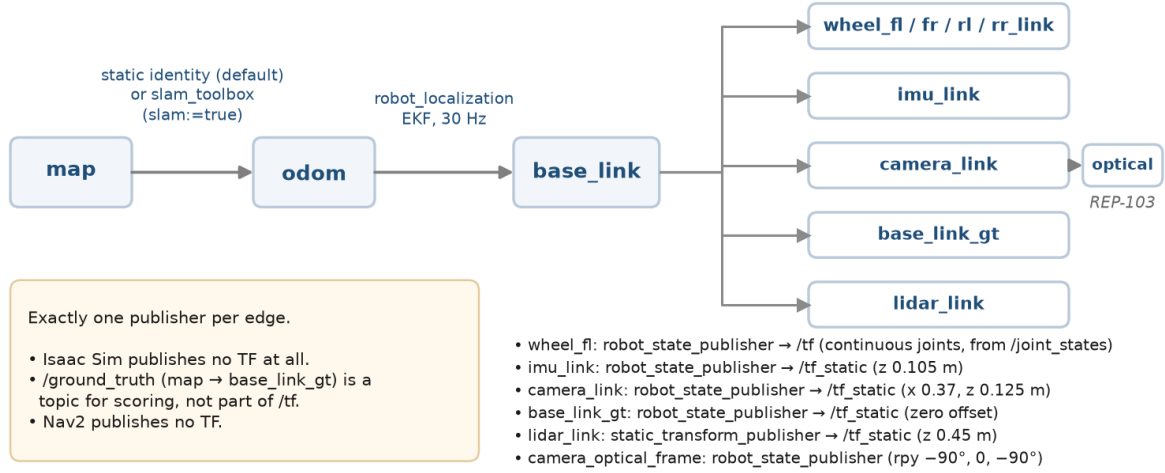


Figure 6. TF tree with the sole owner of each edge.

4.4 Time, lifecycle and rates

Every node runs on simulation time (use_sim_time: true) driven by /clock, so the stack behaves identically at any real-time factor. Nav2 servers are lifecycle nodes brought up in order by lifecycle_manager_navigation (autostart). All Nav2 transform_tolerance values are 1.0 s because a full GUI plus RViz on one GPU drops the simulator to $\approx 0.33\times$ real time. Figure 7 lists the loop rates.

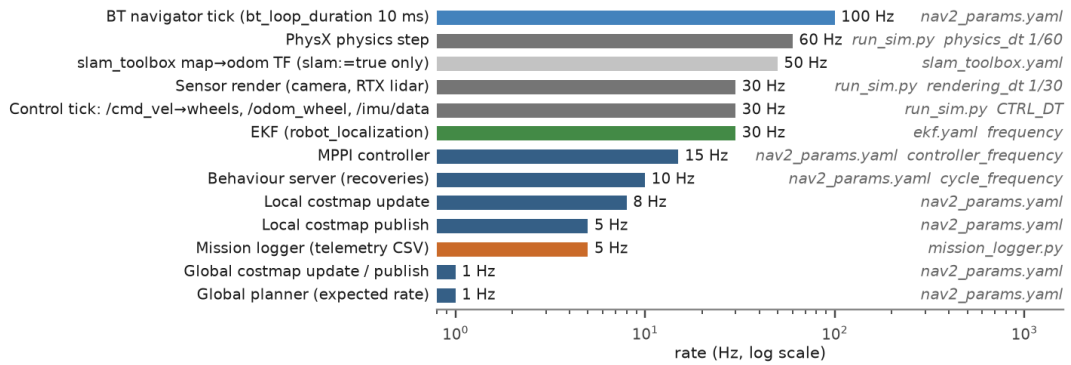


Figure 7. Loop rates of the running system (simulation time), with the file that sets each.

5 Georeferencing without GPS: UTM and the surveyed datum

The operator thinks in UTM; the robot plans in a local metric map frame. The bridge between them is a single **surveyed site datum** — one latitude/longitude fixed at survey time, like a benchmark pillar on a test range (Avadi, Tamil Nadu: 13.114700 °N, 80.109800 °E). It is a constant of the mission, not a measurement made by the vehicle.

Universal Transverse Mercator

UTM divides the Earth into 60 zones 6° wide and projects each with a transverse Mercator projection onto a metric grid. The zone number follows from the longitude λ :

$$\text{zone} = \left\lfloor \frac{\lambda + 180^\circ}{6^\circ} \right\rfloor + 1 = \left\lfloor \frac{260.11}{6} \right\rfloor + 1 = 44 \quad (1)$$

so the site is in **zone 44 N, EPSG:32644** (WGS 84 / UTM 44N, central meridian 81 °E). pyproj converts the datum once, offline, to the anchor $(E_0, N_0) = (403\,506.52, 1\,449\,990.12)$ m, which is written into world.json. At run time the conversion between UTM and the map frame is a translation:

$$x = E - E_0, \quad y = N - N_0, \quad E = x + E_0, \quad N = y + N_0 \quad (2)$$

Over a 150 m site this is exact to within the projection's local distortions: the grid convergence here is $\gamma \approx (\lambda - \lambda_0) \sin \varphi \approx -0.89^\circ \times \sin 13.1^\circ \approx -0.2^\circ$, absorbed by defining the map frame's +y axis as *grid* north; the point scale factor $k \approx 0.9997$ changes a 150 m distance by ≈ 4.3 cm. The round trip lat/lon → UTM → map → UTM → lat/lon was measured at 0.000 mm.

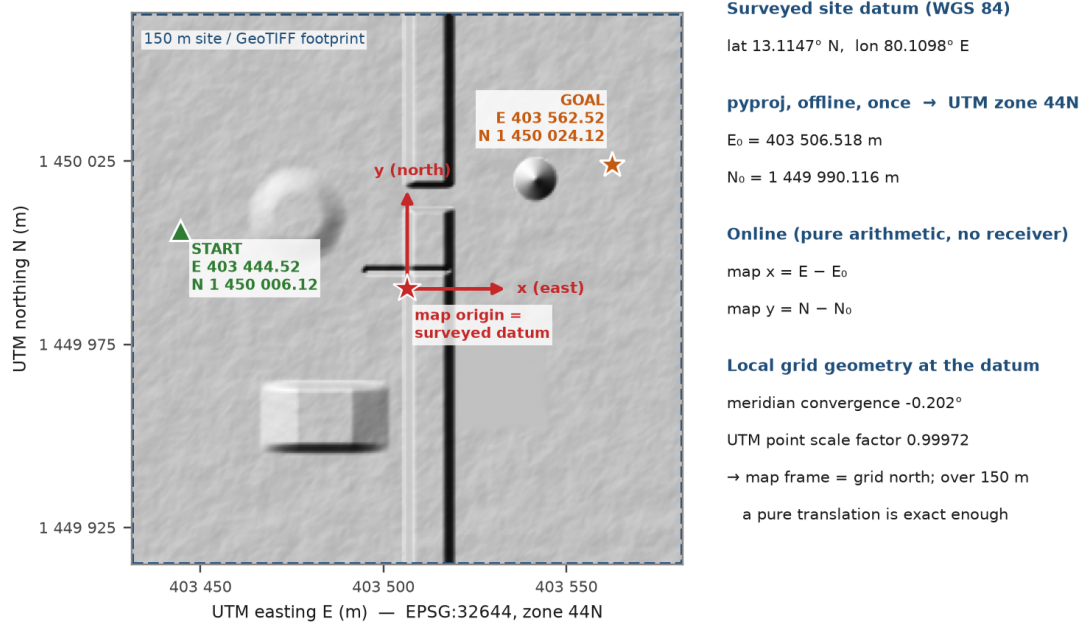


Figure 8. The site in UTM zone 44N: surveyed datum (map origin), start and default goal with their UTM coordinates, and the 150 m $\times$ 150 m extent of the prior DEM.

utm_goal.py implements the operator interface: it reads the anchor from world.json, computes $(x, y) = (E - E_0, N - N_0)$, rejects goals outside the ± 75 m site, sends a NavigateToPose goal in frame *map*, and prints feedback in both frames (map metres and UTM) with the distance remaining and the recovery count. The committed goal E 403 562.52 N 1 450 024.12 maps to (56.00, 34.00) m; the start (-62, 16) m is E 403 444.52, N 1 450 006.12. Nothing at run time imports pyproj, and nothing subscribes to a position fix.

6 The digital elevation model (DEM)

A DEM is a raster whose cells store ground elevation. It is the robot's prior knowledge of the terrain: which slopes are too steep, where the ditch and the berm are, where the tunnel passes. In the field it comes from a survey or a national product (the QSLAM design preloads ISRO Bhuvan / CartoDEM tiles); in this workspace it is synthesised analytically so that the simulator and the planner can be built from the same numbers.

Terrain synthesis

The height at map coordinates (x, y) is a sum of a fractal micro-relief and parametric landforms:

$$h(x, y) = m_{\text{tex}} h_{\text{fBm}} + h_{\text{ridge}} + h_{\text{mound}} + h_{\text{ditch}} + h_{\text{berm}} (1 - c_{\text{tunnel}}) + h_{\text{plateau}} \quad (3)$$

The micro-relief is fractional Brownian motion built from four octaves of hashed-lattice value noise $n(\cdot)$, each octave halving the amplitude and shrinking the lattice cell by 0.45:

$$h_{\text{fBm}}(\mathbf{p}) = \sum_{k=0}^3 a_k n(\mathbf{p}/c_k), \quad a_k = 0.06 \cdot 0.5^k \text{ m}, \quad c_k = 6 \cdot 0.45^k \text{ m} \quad (4)$$

Each noise value is bilinearly interpolated between four lattice values with the smoothstep weight $S(t) = 3t^2 - 2t^3$, which also shapes every landform edge so the surface has no creases. The tunnel is carved by multiplying the berm by $(1 - c)$, where c is a smooth box profile across $y = 25$ m; `build_scene.py` then roofs the carved bore (roof 3.0–3.6 m) in the USD stage.

Table 6. Landforms and objects of the seed-42 proving ground.

Feature	Parameters (map frame, metres)	Role in the mission
Earth berm + tunnel	x 2-10, full site width, 3.2 m high; bore at y = 25, half-width 2.8 m	Wall across the site: the tunnel is the only way through
Plateau	centre (-30, 20), top radius 6, 1.1 m, 8° flanks	Drivable slope on the route

Feature	Parameters (map frame, metres)	Role in the mission
Mound	cone at (35, 30), 4 m, 35°	Must be skirted (> 15°)
Ridge	x -40...-5, y -45...-25, 12° faces	Ramp-plateau-ramp
Ditch	y = 5, x -12...12, 1.5 m deep, 2 m wide	Negative obstacle only the prior can reveal
Low-texture patch	x 10-40, y -40...-10	Suppressed relief: stress case for visual odometry
Trees / rocks	40 trunks at (-45, 45); 60 rocks at (55, -55)	Scene clutter
Corridor obstacles	14 drums, crates, logs, boulders along the route	Absent from the prior — found by lidar
Humans	2 standing, 2 patrols at 0.8 and 0.7 m/s crossing the corridor	Dynamic obstacles, absent from the prior

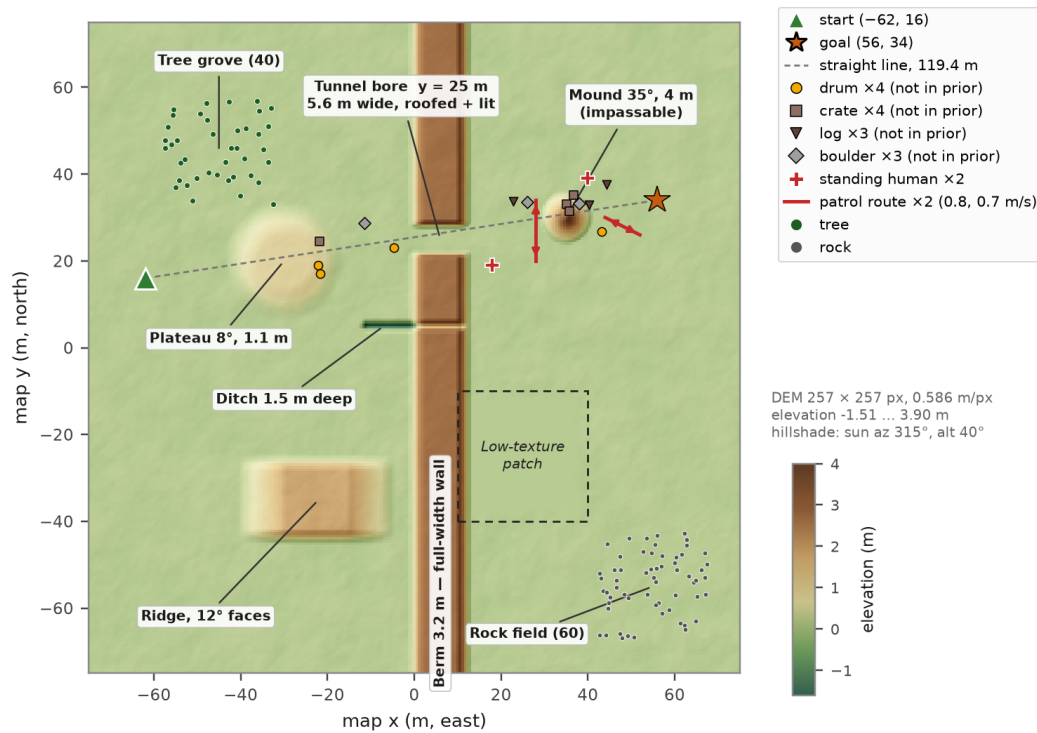


Figure 9. The prior DEM (257 × 257 px over 150 m, 0.586 m/px) with every landform, the corridor obstacles and humans that the prior deliberately does not contain, start and goal.

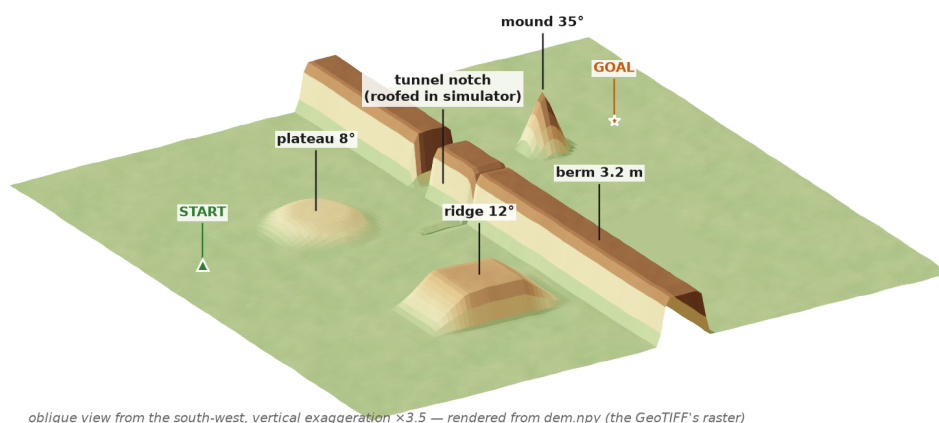


Figure 10. Shape of the terrain: berm with the tunnel notch, the 35° mound and the 8° plateau.

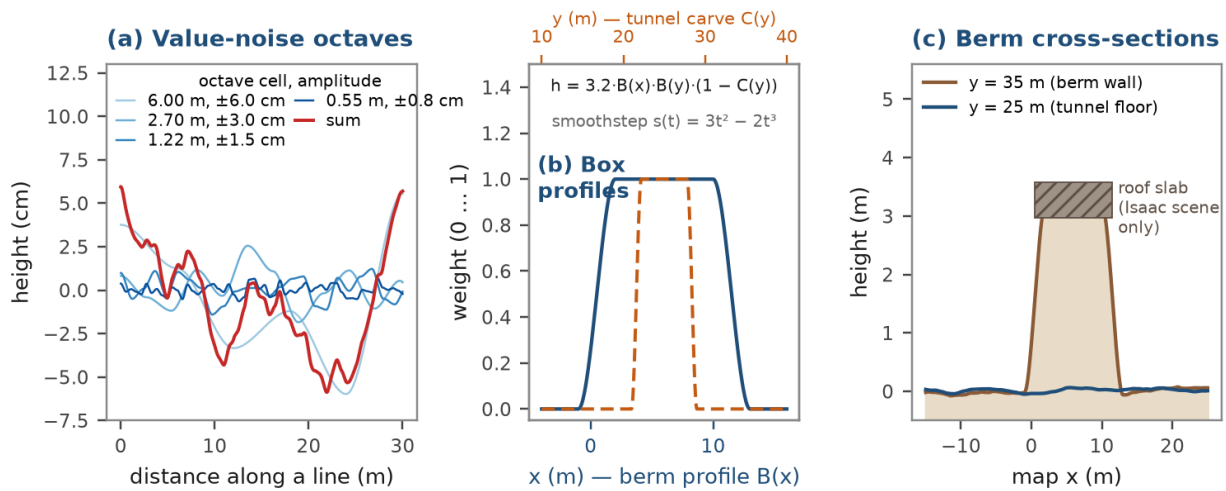


Figure 11. Terrain synthesis: (a) value-noise octaves and their fBm sum; (b) smoothstep and the box profile used for landform edges; (c) cross-sections through the tunnel and through the solid berm.

7 GeoTIFF – the single source of truth

A GeoTIFF is an ordinary TIFF image plus GeoKeys that state its coordinate reference system and how pixel indices map to ground coordinates. world.py writes the DEM as `prior_dem_utm44n.tif`: one float32 band “terrain elevation above site datum (m)”, 257×257 pixels, CRS EPSG:32644, deflate-compressed in 256×256 tiles, with the datum stored as tags (SITE_LAT/LON, ANCHOR_E/N). It opens in QGIS like any survey product.

Affine geotransform

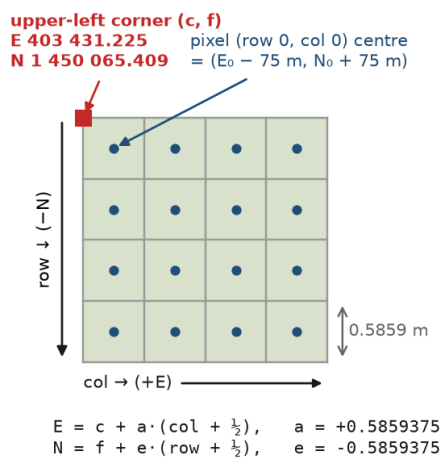
Pixel (col, row) maps to UTM through a six-parameter affine transform; for a north-up raster the rotation terms are zero:

$$E = c + a \text{ col}, \quad N = f + e \text{ row}, \quad a = 0.5859 \text{ m}, \quad e = -0.5859 \text{ m} \quad (5)$$

with $(c, f) = (403\,431.225, 1\,450\,065.409)$ the upper-left corner of the upper-left pixel – half a pixel beyond the outermost sample (“pixel is area”). Row 0 is the north edge. The inverse used when sampling, with bilinear interpolation between the four surrounding pixels (fractional offsets f_r, f_c):

$$\text{col} = \frac{E - c}{a} - \frac{1}{2}, \quad \text{row} = \frac{f - N}{e} - \frac{1}{2}, \quad h = (1 - f_r)[(1 - f_c)h_{00} + f_c h_{01}] + f_r[(1 - f_c)h_{10} + f_c h_{11}] \quad (6)$$

(a) Raster $\leftrightarrow$ UTM georeferencing



(b) Read back from prior_dem_utm44n.tif

```
driver      GTiff (1 band)
size        257 x 257 px
dtype       float32 - elevation (m)
CRS         EPSG:32644
            WGS 84 / UTM zone 44N
transform   a=0.5859375 b=0
            c=403 431.225
            d=0 e=-0.5859375
            f=1 450 065.409
bounds E    403 431.2 ... 403 581.8
bounds N    1 449 914.8 ... 1 450 065.4
storage     DEFLATE, tiled 256x256
band        terrain elevation above site datum (m)
SITE_LAT/LON 13.114700 / 80.109800
ANCHOR_E    403506.518
ANCHOR_N    1449990.116
SOURCE      SIH26126 prior survey DEM
pixels      identical to dem.npy
```

Figure 12. GeoTIFF structure: (a) the raster corner, pixel centres and affine transform; (b) the metadata read back from the file with rasterio.

Why one file feeds everything

The same GeoTIFF produces (1) the Isaac Sim terrain mesh (build_scene.py), (2) the Nav2 occupancy map (§8) and (3) the UTM anchor. The simulated ground and the planner's belief therefore agree by construction, and a whole class of “map does not match the world” bugs cannot occur. In the MuJoCo prototype the planner reads the GeoTIFF back off disk through rasterio (class PriorDEM) rather than touching simulator state, so the file path is genuinely exercised. In the field the same interface accepts a surveyed or satellite-derived DEM.

8 Traversability and the prior occupancy map

Traversability is derived from the slope of the DEM. Using central differences at the DEM's own resolution (a coarser stencil smooths steep faces into something that looks drivable):

$$\theta(x, y) = \arctan \sqrt{\left(\frac{\partial h}{\partial x}\right)^2 + \left(\frac{\partial h}{\partial y}\right)^2}, \quad \text{cell blocked} \Leftrightarrow \theta > 15^\circ \quad (7)$$

15° is a conservative limit for a 60 kg skid-steer on loose soil: the 8° plateau remains drivable, while the 35° mound, the ditch walls and the berm faces are blocked. The result is written as map.pgm (0 = occupied, 254 = free) with map.yaml: mode trinary, resolution 0.585938 m, origin (−75.293, −75.293), occupied_thresh 0.65, free_thresh 0.25. map_server publishes it latched on /prior_map in frame map; 5.3 % of the site is blocked. Obstacles and people are deliberately absent — the rover must discover them.

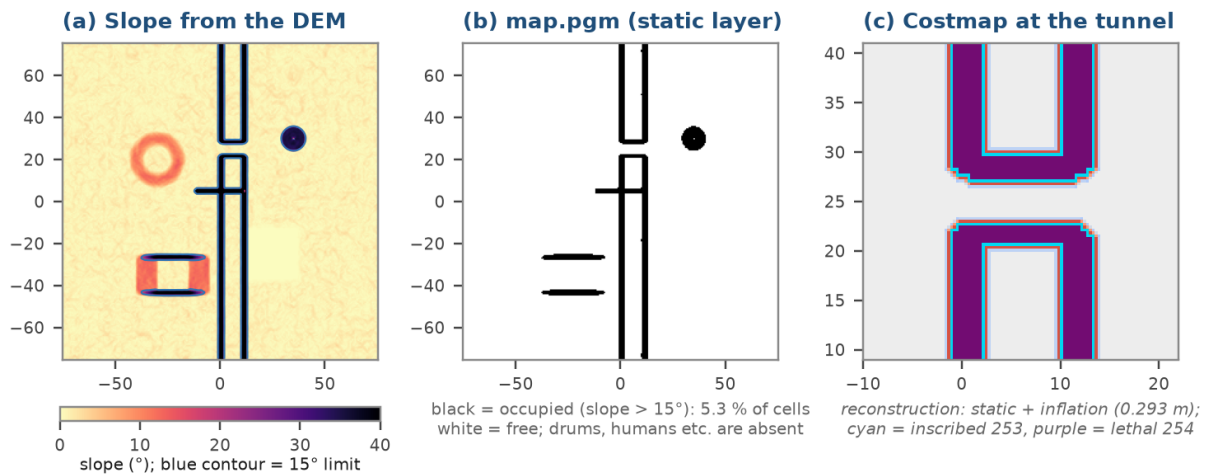


Figure 13. From elevation to cost: (a) slope with the 15° threshold; (b) the occupancy map Nav2 loads; (c) the resulting global costmap after inflation (reconstruction).

9 Costmaps

Nav2 plans on **layered costmaps**: 2-D grids whose cells hold a cost 0–254, assembled from plugin layers. QSLAM uses two.

Table 7. The two costmaps.

	Global costmap	Local costmap
Frame / window	map, whole site	odom, rolling 14 × 14 m around the robot
Resolution	0.293 m (half a DEM pixel)	0.15 m
Layers	static (/prior_map) + obstacle (/scan) + inflation	obstacle (/scan) + inflation
Update / publish	1 Hz / 1 Hz	8 Hz / 5 Hz
Used by	Smac 2D global planner	MPPI controller, behaviours

Obstacle layer. Each /scan beam is ray-traced through the grid (Bresenham line from the lidar origin): cells along the beam are *cleared* up to 20 m, the end cell is *marked* lethal up to 18 m, for returns below 2.5 m height. Clearing is what lets a walking person leave no ghost behind. **Inflation layer.** The robot is modelled as a disc of radius $r = 0.55$ m; costs spread outward from every lethal cell as a function of the distance d :

$$c(d) = 252 e^{-k(d-r)} \quad \text{for } r < d \leq R, \quad c = 253 \text{ for } d \leq r, \quad c = 254 \text{ at the obstacle}, \quad k = 3.0, \quad R = 0.95 \text{ m} \quad (8)$$

A cell of cost 253 means the robot's centre there would put the body in collision; the exponential band lets planners trade clearance against distance (Figure 14).

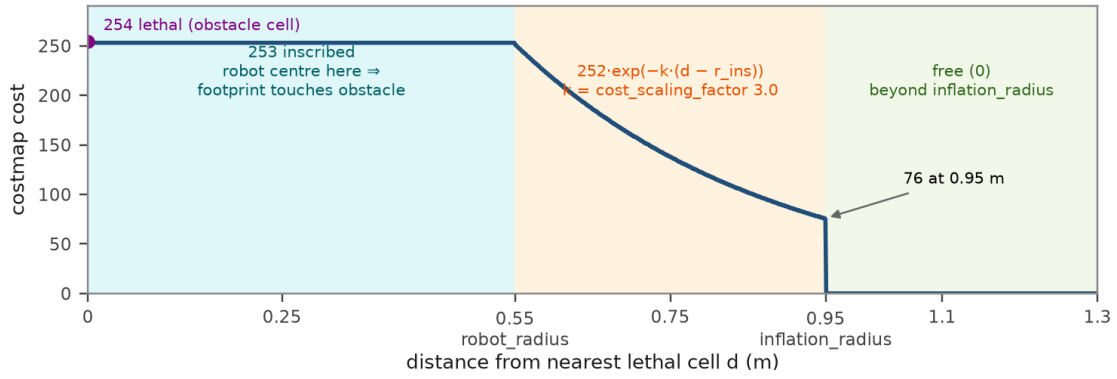


Figure 14. Inflation cost as a function of distance from an obstacle for the configured robot radius 0.55 m, cost_scaling_factor 3.0 and inflation radius 0.95 m.

10 Global planner – A* (Nav2 Smac Planner 2D)

10.1 How A* works

A* finds the cheapest path on a graph – here, grid cells connected to their 8 neighbours. It keeps an **open set** ordered by

$$f(n) = g(n) + h(n) \quad (9)$$

where $g(n)$ is the exact cost from the start to cell n and $h(n)$ an estimate of the remaining cost. Each iteration pops the cell with the lowest f , moves it to the **closed set**, and relaxes its neighbours: if $g(n) + \text{cost}(n, m)$ improves $g(m)$, the neighbour's parent and priority are updated. When the goal is popped the path is read back through the parents. If h never over-estimates (admissible) and satisfies $h(n) \leq \text{cost}(n, m) + h(m)$ (consistent), the first path found is optimal and no cell is expanded twice. The Euclidean distance to the goal satisfies both.

10.2 Smac Planner 2D in QSLAM

Smac 2D is Nav2's cost-aware A* on the global costmap (Moore 8-neighbourhood, Euclidean heuristic). The cost of stepping to a neighbour grows with that cell's costmap value $c \in [0, 252]$:

$$\text{cost}(n, m) = \ell_{nm} \left(1 + \mu \frac{c(m)}{252}\right), \quad \ell_{nm} \in \{1, \sqrt{2}\}, \quad \mu = \text{cost_travel_multiplier} = 1.0 \quad (10)$$

Lethal and inscribed cells (≥ 253) are not expanded. Configuration: tolerance 0.5 m, allow_unknown true, max_iterations 10^6 , max_planning_time 3.0 s, no costmap down-sampling, final approach orientation kept. The behaviour tree re-plans at 1 Hz, so newly discovered obstacles in the global costmap re-route the path within a second.

The tunnel lesson ($\mu = 1.0$). The 5.6 m bore leaves only a narrow lane free of inscribed cost, and every cell in it carries inflation cost that μ multiplies along the whole bore. During tuning on the live stack, values above 1.0 made the planner refuse the tunnel, so μ is pinned at 1.0. The offline re-computation in Figure 15 uses only the static prior: there, because the berm spans the entire site, $\mu = 1, 2$ and 4 all return the same 125.8 m tunnel route (23 007 expansions at $\mu = 1$; straight line 119.4 m) – the refusal therefore came from live-costmap conditions this reconstruction does not include.

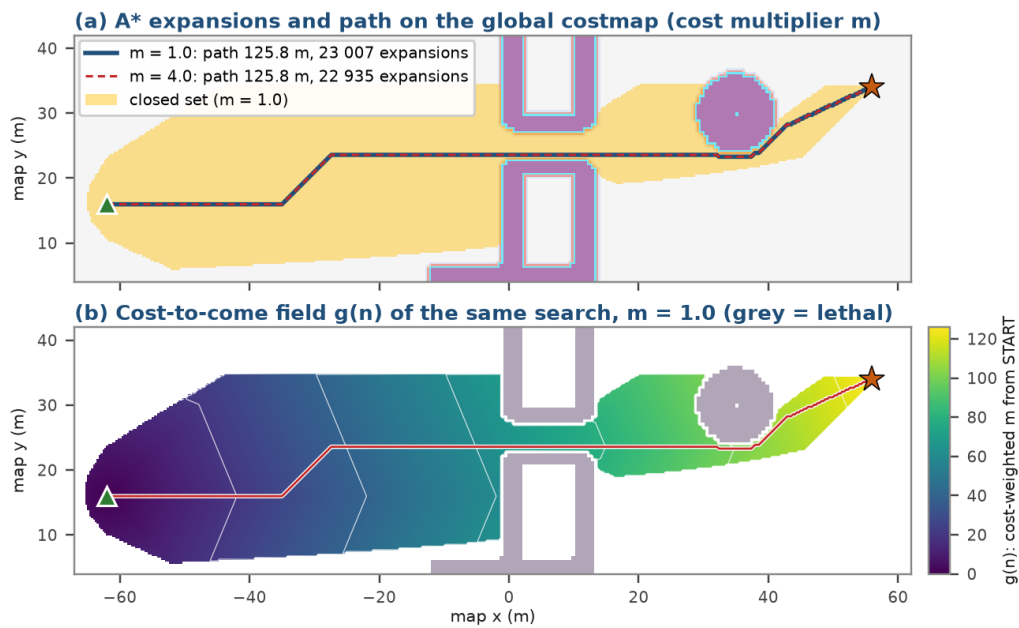


Figure 15. A* on the reconstructed global costmap (textbook implementation of the Smac 2D cost model, run offline): expanded cells and the resulting route through the tunnel.

10.3 Hybrid-A* — the QSLAM target planner QSLAM DESIGN · planned

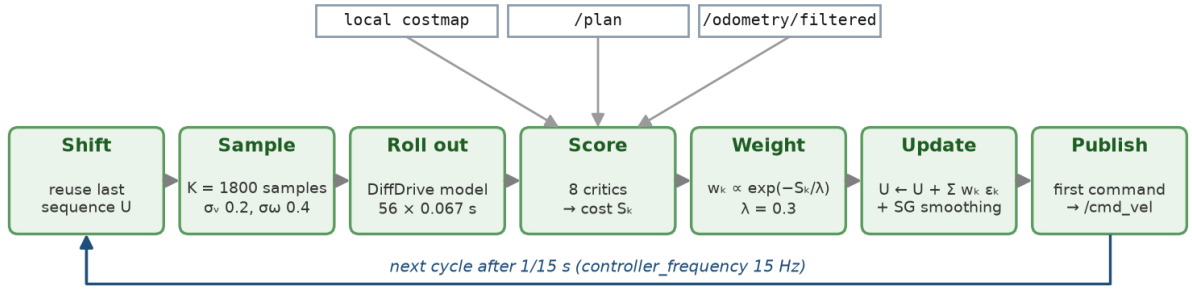
For a larger vehicle that cannot pivot in place, QSLAM specifies Smac Hybrid-A*. The search state becomes (x, y, ψ) with the heading discretised (typically 72 bins); successors are generated by short kinematically feasible motion primitives — straight segments and arcs of the minimum turning radius (Dubins, or Reeds–Shepp when reversing is allowed) — rather than grid steps. The heuristic is the maximum of a 2-D obstacle-aware distance and the obstacle-free non-holonomic distance, and an analytic expansion tries to connect directly to the goal with a Reeds–Shepp curve whenever it is near. The result is a path the vehicle can actually track. The skid-steer testbed rover can turn on the spot, so the 2-D planner is sufficient there.

10.4 Prototype planner (MuJoCo) IMPLEMENTED · MuJoCo prototype

The prototype runs its own A* on a 1 m grid sampled from the GeoTIFF, with blocked cells dilated by the footprint plus a 0.20 m margin and a soft terrain cost that prefers flat ground: $\text{cost} = \ell \cdot (1 + 2.5 (\theta/15^\circ)^2)$. It also builds a metric distance field to non-traversable terrain, the channel through which the ditch — a hole the camera cannot see as an obstacle — reaches the local planner.

11 Local planner — Model Predictive Path Integral control (MPPI)

The global path ignores what the lidar has just discovered and what the wheels can do in the next second. The local planner closes that gap. QSLAM uses Nav2's MPPI controller, a sampling-based model predictive controller derived from information-theoretic optimal control (Williams et al., 2017). Instead of solving an optimisation analytically it **imagines thousands of futures, scores them, and averages the good ones.**



Critic weights: Constraint 4.0 · Cost 3.81 · Goal 5.0 · GoalAngle 3.0 · PathAlign 14.0 · PathFollow 5.0 · PathAngle 2.0 · PreferForward 5.0

Limits: $v_x \in [-0.25, 0.90]$ m/s, $|\omega_z| \leq 1.0$ rad/s · $\gamma = 0.015$ · motion model DiffDrive · visualize: true (/trajectories)

Figure 16. One MPPI control cycle, repeated at 15 Hz.

11.1 The algorithm

The controller keeps a nominal control sequence $U = (u_0 \dots u_{T-1})$ with $u = (v_x, \omega_z)$, $T = 56$ steps of $\Delta t = 0.067$ s – a 3.75 s horizon. Each cycle:

- **Sample** $K = 1800$ perturbed sequences, $\epsilon_{k,t} \sim N(0, \Sigma)$, $\Sigma = \text{diag}(0.2^2, 0.4^2)$, and clamp them to $v_x \in [-0.25, 0.9]$ m/s, $|\omega_z| \leq 1.0$ rad/s.
- **Roll out** each sequence through the differential-drive model:

$$x_{t+1} = x_t + v_t \cos \psi_t \Delta t, \quad y_{t+1} = y_t + v_t \sin \psi_t \Delta t, \quad \psi_{t+1} = \psi_t + \omega_t \Delta t \quad (11)$$

- **Score** every trajectory τ_k with the weighted critics (critic table below) plus a control-cost term that keeps the update consistent with the sampling distribution:

$$S_k = \sum_i w_i C_i(\tau_k) + \gamma \sum_{t=0}^{T-1} \mathbf{u}_t^T \Sigma^{-1} \epsilon_{k,t}, \quad \gamma = 0.015 \quad (12)$$

- **Weight** the samples with a softmax of their cost (temperature $\lambda = 0.3$) and **update** the sequence:

$$w_k = \frac{\exp(-\frac{1}{\lambda}(S_k - S_{\min}))}{\sum_{j=1}^K \exp(-\frac{1}{\lambda}(S_j - S_{\min}))}, \quad \mathbf{U} \leftarrow \mathbf{U} + \sum_{k=1}^K w_k \epsilon_k \quad (13)$$

- **Smooth and act:** Nav2 applies a 9-point quadratic Savitzky–Golay filter to the sequence, publishes u_0 on /cmd_vel, shifts the sequence one step and repeats.

A small λ makes the average follow only the very best samples; a large λ averages broadly. Because every sample is a whole time-varying sequence, MPPI can find manoeuvres such as “swing left, then straighten” that a constant-arc sampler cannot, and it needs no gradient of the cost, so non-convex costs such as costmaps are handled naturally. In Figure 17, 119 of the 1 800 samples carry 90 % of the weight.

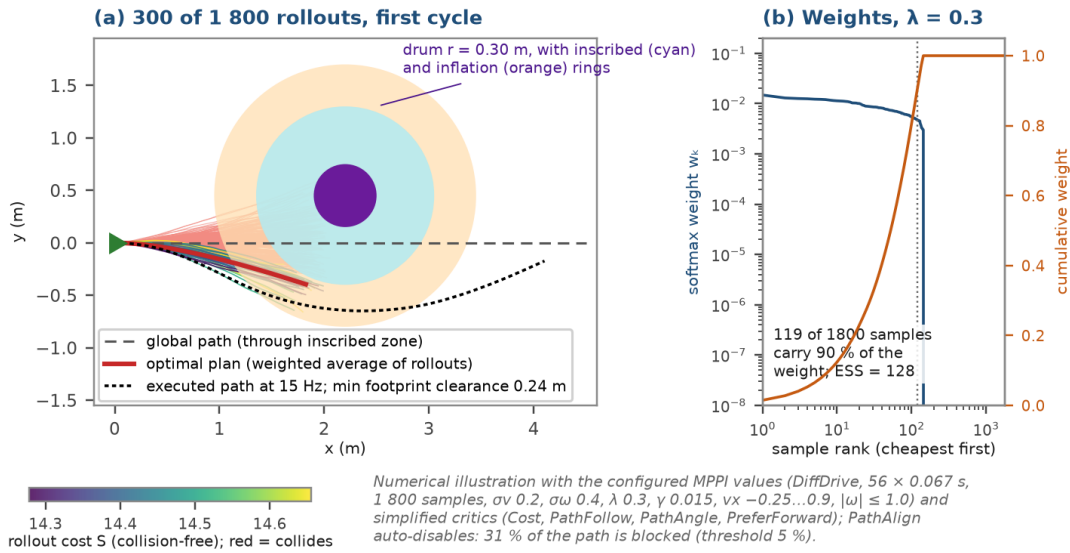


Figure 17. MPPI around an obstacle (numerical illustration with the configured horizon, batch size, noise and temperature; simplified critic set): (a) sampled rollouts coloured by cost and the weighted result; (b) how few samples carry most of the weight.

Table 8. MPPI critics and weights (nav2_params.yaml).

Critic	Weight	What it scores
ConstraintCritic	4.0	Violations of the velocity limits of the motion model
CostCritic	3.81	Costmap cost along the trajectory; collision cost 10^6 , critical cost 300; circular footprint
GoalCritic	5.0	Distance of the trajectory end to the goal (active within 1.4 m)
GoalAngleCritic	3.0	Heading towards the goal (within 0.5 m)
PathAlignCritic	14.0	Alignment of trajectory points with the path; switched off when > 5 % of the path is blocked, so the robot may leave it to pass an obstacle
PathFollowCritic	5.0	Progress towards a point 5 poses ahead on the path
PathAngleCritic	2.0	Heading deviating more than 1.0 rad from the path direction
PreferForwardCritic	5.0	Reverse motion (allowed, but discouraged)

Around the controller: controller_frequency 15 Hz; progress checker requires 0.5 m of movement in 20 s; goal checker accepts 0.7 m with any final heading (yaw tolerance 6.28 rad, stateful); failure_tolerance 0.5 s. A stale-command watchdog in run_sim.py stops the wheels if no /cmd_vel arrives for 0.5 s.

11.2 Prototype local planner – DWA IMPLEMENTED · MuJoCo prototype

The MuJoCo prototype uses the Dynamic Window Approach: it samples constant (v, ω) pairs (6 speeds up to the acceleration-limited window plus two reverse speeds, 25 turn rates), rolls each out for 2.4 s at 0.2 s steps, rejects any whose clearance to the depth-built costmap or to prior terrain hazards is below 0.65 m, and minimises

$$J = 2.4 d_{\text{goal}} + 4 \frac{\max(0, r_s - c)}{r_s} + 1.4 (v_{\text{max}} - v) + 0.3 |\omega| + 1.1 \min(\max(o - 4, 0), 10) + 2.5 [v < 0] + 3 [|v| < 0.05] \quad (14)$$

with $r_s = 0.75$ m the soft clearance, c the rollout's minimum clearance and o the offset from the surveyed route. Making the margin soft (not hard) is what prevents the classic DWA deadlock where one noisy cell rejects every sample.

12 Behaviour tree, supervision and recovery

bt_navigator executes Nav2's default *navigate_to_pose_w_replanning_and_recovery* behaviour tree (Figure 18). A PipelineSequence runs ComputePathToPose under a 1 Hz RateController and FollowPath continuously; each has its own single-retry recovery that clears the corresponding costmap. If both fail, the outer RecoveryNode (6 retries) runs a RoundRobin of recovery behaviours, one per failure: clear both costmaps → spin 1.57 rad → wait 5 s → back up 0.30 m at 0.15 m/s. A new goal pre-empts recovery (GoalUpdated).

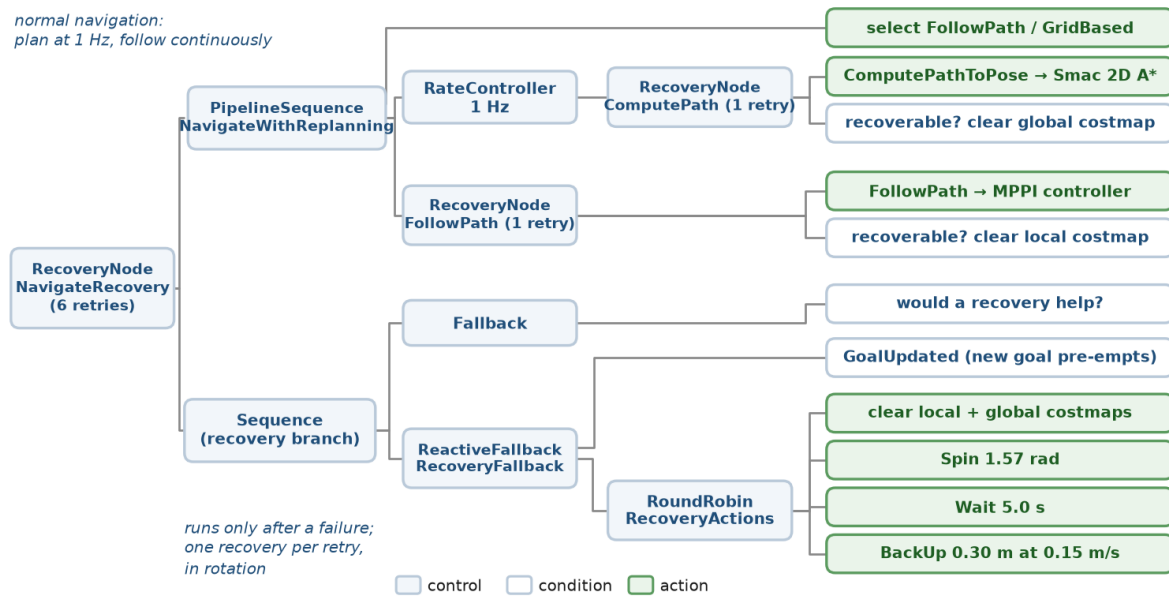


Figure 18. The navigate-to-pose behaviour tree used by QSLAM (Nav2 Jazzy default).

behavior_server provides spin, backup, drive_on_heading and wait at 10 Hz with rotational limits 0.2–0.8 rad/s and 2.0 rad/s², checking 2.0 s ahead against the local costmap. The committed and run_1080p missions needed one recovery each; run_final needed seven, and Figure 19 reconstructs them from nav.log: the rover stalled 1 m from an unsurveyed crate (five no-progress aborts), a spin freed it, and two later controller aborts were cleared before the goal succeeded at 986 s. Short reverse commands (amber) are MPPI's own choice, allowed down to -0.25 m/s.

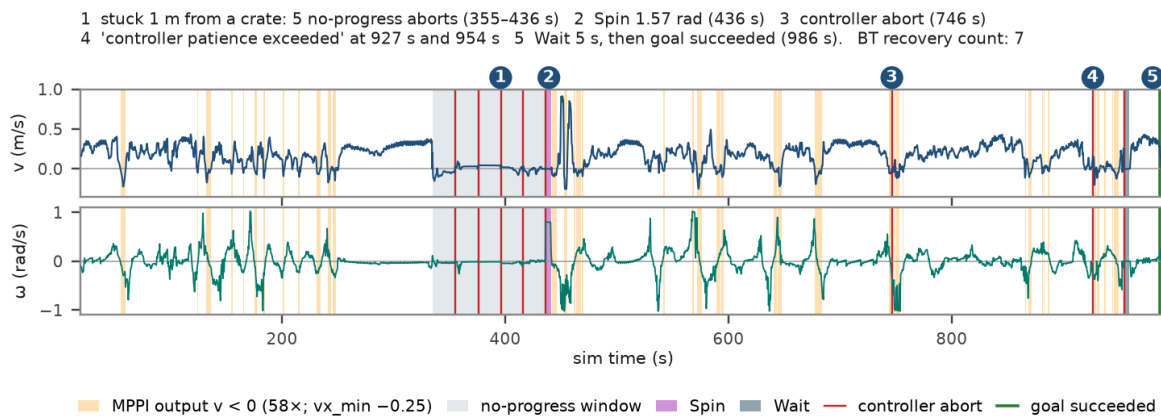


Figure 19. Velocity commands during the 189 m run (run_final) with the recovery events taken from nav.log; amber = MPPI reverse outputs, grey = no-progress window.

13 Localization – the Extended Kalman Filter

Without GPS, “where am I?” is the hardest question. QSLAM answers it with robot_localization's EKF, the sole owner of odom→base_link. A Kalman filter keeps a Gaussian belief — a mean state $\hat{x}$ and covariance P — and alternates **prediction** (move the belief with a motion model, grow the uncertainty) and **update** (correct it with each measurement, weighted by relative confidence). The *extended* filter linearises the nonlinear models with Jacobians (Figure 20).

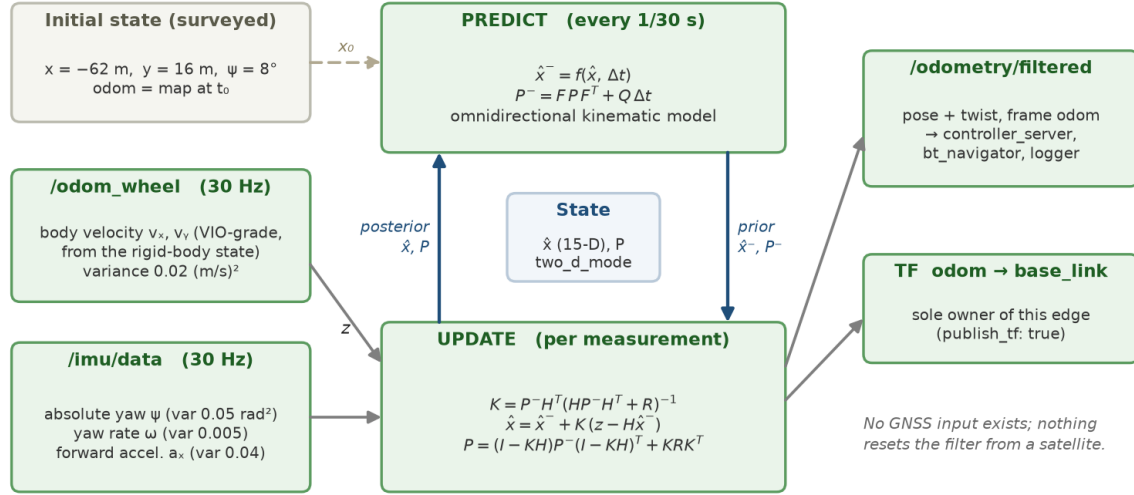


Figure 20. EKF predict/update cycle as configured in config/ekf.yaml.

13.1 State and prediction

The state has 15 elements — position, orientation, linear and angular velocity, linear acceleration: $x = [x \ y \ z \ \varphi \ \theta \ \psi \ v_x \ v_y \ v_z \ \omega_x \ \omega_y \ \omega_z \ a_x \ a_y \ a_z]^T$. With two_d_mode the vertical, roll and pitch components are held at zero. The filter runs at 30 Hz with an omnidirectional kinematic model; for the planar part:

$$x_{k+1} = x_k + (v_x \cos \psi - v_y \sin \psi) \Delta t, \quad y_{k+1} = y_k + (v_x \sin \psi + v_y \cos \psi) \Delta t, \quad \psi_{k+1} = \psi_k + \omega_z \Delta t \quad (15)$$

(plus $\frac{1}{2}a\Delta t^2$ terms and $v \leftarrow v + a\Delta t$). The covariance is propagated with the Jacobian $F = \partial f / \partial x$ and the process noise Q :

$$\hat{x}_{k+1}^- = f(\hat{x}_k, \Delta t), \quad P_{k+1}^- = F P_k F^T + Q \Delta t \quad (16)$$

13.2 Measurement update

Each sensor message selects the state elements it measures (a boolean mask per sensor, which forms H) and carries its own covariance R . The update computes the innovation ν , its covariance S and the Kalman gain K , and uses the Joseph form for numerical stability:

$$\nu = z - H\hat{x}^-, \quad S = H P^- H^T + R, \quad K = P^- H^T S^{-1} \quad (17)$$

$$\hat{x} = \hat{x}^- + K\nu, \quad P = (I - KH) P^- (I - KH)^T + KRK^T \quad (18)$$

Table 9. EKF inputs (config/ekf.yaml, covariances from run_sim.py).

Input	Fused state elements	Covariance (R)	Physical source
odom0: /odom_wheel	v_x, v_y (body frame)	0.02 (m/s) ²	VIO-grade body velocity (cuVSLAM on hardware); synthesised from the rigid-body state in the testbed
imu0: /imu/data	ψ (absolute yaw), ω_z, a_x	ψ 0.05 rad ² , ω_z 0.005, a_x 0.04	AHRS / magnetometer-grade heading, gyro, accelerometer (gravity removed)
initial_state	$x = -62.0, y = 16.0, \psi = 0.1396$ rad (8°)	—	Surveyed deployment pose (launch point)

13.3 Why these inputs – a measured decision

Wheel-encoder odometry was tried first and failed in three measured ways on this skid-steer: (1) yaw from the left/right wheel-speed difference is meaningless when the wheels scrub sideways in every turn; (2) distance over-counts by about 30 % on slopes because the wheels slip; (3) sideways crabbing on side slopes is invisible to encoders. Missions ended 10–70 m from where the robot believed it was. Integrating the gyro alone for heading drifted about 13° over 45 m. The fused quantities are therefore exactly what a visual-inertial odometry system and an AHRS measure on the real vehicle. Figure 21 reconstructs, from the recorded 189 m path, how each failure mode grows: gyro-only heading ends 57 m off, a 30 % wheel over-count on slopes 4.3 m, the adopted EKF 1.09 m.

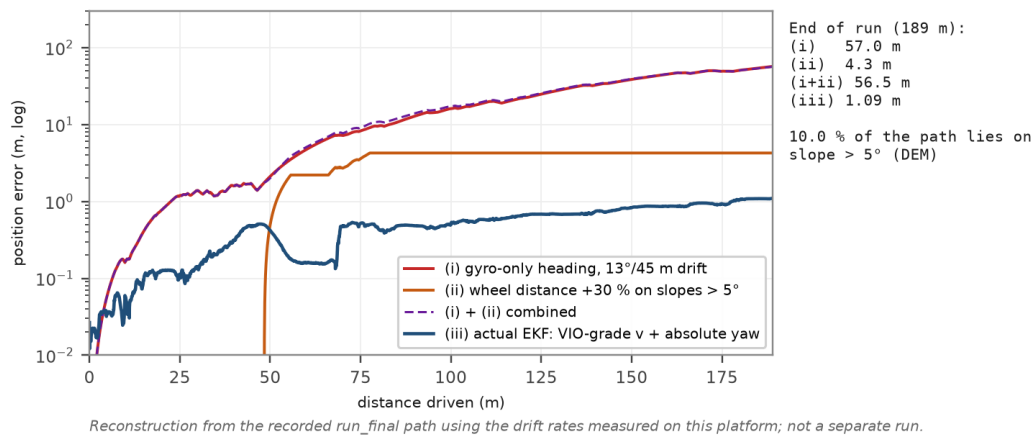


Figure 21. Position error versus distance: gyro-only heading and slipping wheel odometry (reconstructed from the recorded ground-truth path with the drift rates measured on this platform) against the EKF actually used.

13.4 EKF estimation results

The evaluation logger records ground truth, the EKF estimate and the commands at 5 Hz. Localization error is $\|p_{GT} - p_{EKF}\|$; it is what the robot does not know about its own position. Figure 22 overlays estimate and truth for the two recorded runs and Figure 23 analyses the error.

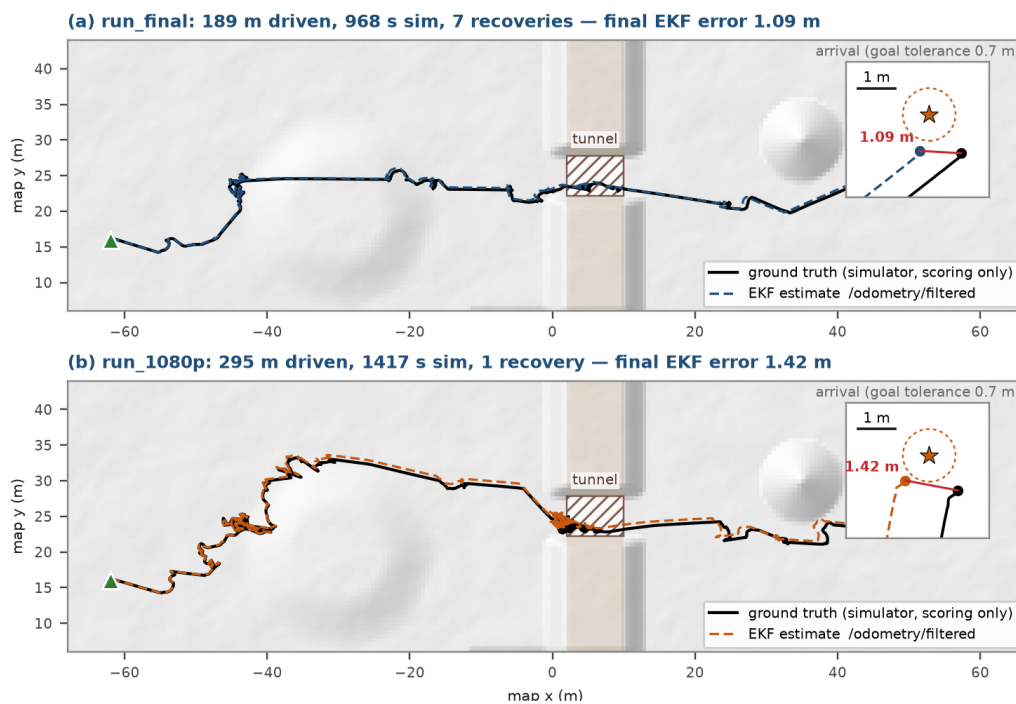


Figure 22. EKF estimate versus ground truth for the two recorded missions. Insets: the final offset at arrival.

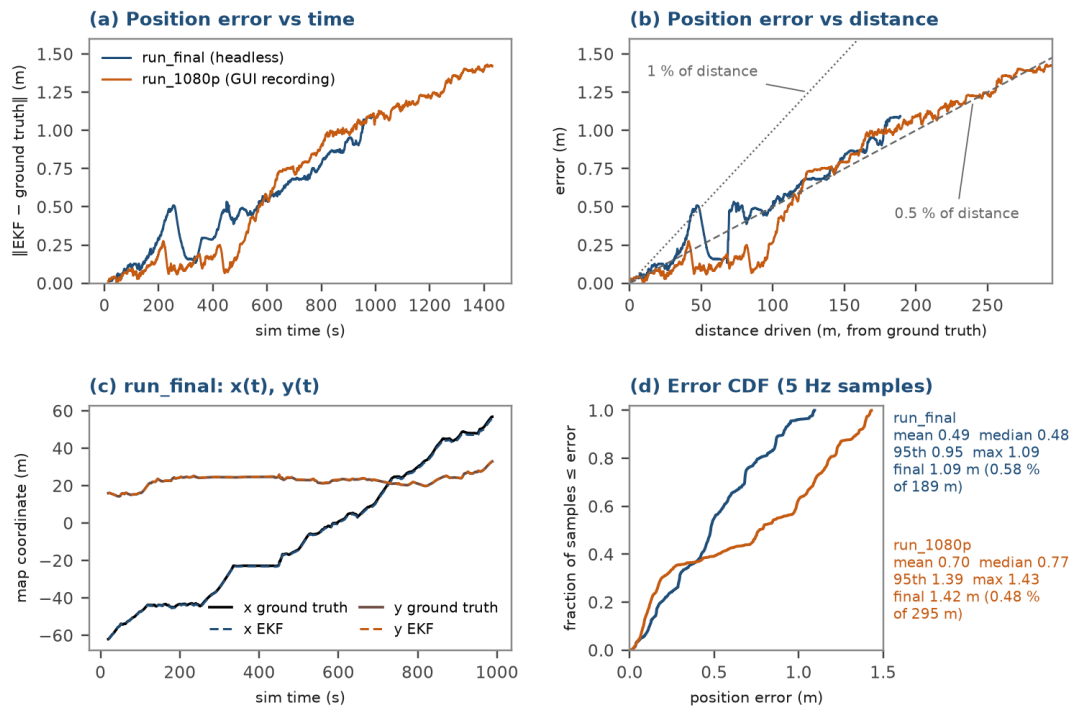


Figure 23. EKF estimation error: (a) over time; (b) against distance travelled; (c) x and y components; (d) distribution.

Table 10. EKF localization error per run (ground truth used for scoring only). Distances are summed from 5 Hz ground truth and include manoeuvring.

Run	Distance	Mean	Median	95th pct	Max	Final	Final / dist.
run_final (headless, 7 recoveries)	189.0 m	0.49 m	0.48 m	0.95 m	1.09 m	1.09 m	0.58 %
run_1080p (GUI recording, 1 recovery)	295.1 m	0.70 m	0.77 m	1.39 m	1.43 m	1.42 m	0.48 %
committed run (README; not re-recorded)	123.5 m plan	—	—	—	—	0.98 m	—

Reading these numbers honestly. The EKF's velocity and heading inputs are synthesised from the simulator's rigid-body state, i.e. idealised VIO/AHRS-grade sensors without their noise. The ≈ 1 m errors therefore measure the integration pipeline – 30 Hz discretisation, message timing, and the 2-D projection of 3-D motion (in two_d_mode a body speed on a slope θ is integrated as horizontal, over-counting by $1/\cos \theta - 1$: 1 % at 8° , 3.5 % at 15°) – not real sensor noise. Replacing the synthesised input with cuVSLAM (Phase 2b) and bounding drift by map anchoring (§15.1) are the designated next steps.

14 Visual SLAM

Visual SLAM (simultaneous localization and mapping) estimates the camera's motion from the images themselves while building a map of landmarks. A **front-end** turns images into relative motion (visual odometry); a **back-end** keeps drift in check by optimising a graph of keyframes and closing loops when a place is recognised. Figure 24 shows the pipeline and which parts come from which engine.

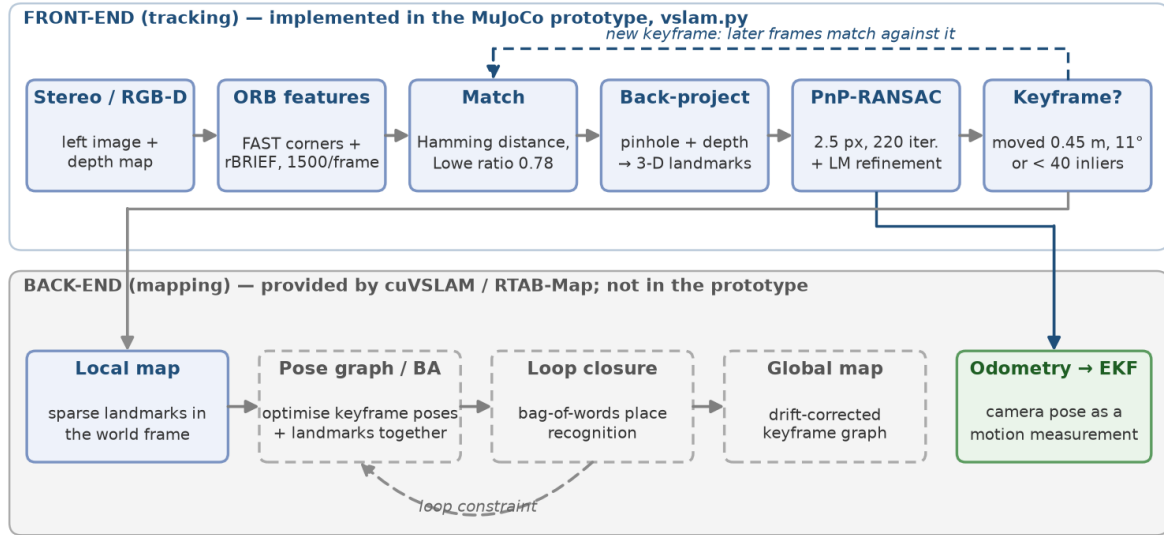


Figure 24. Visual SLAM pipeline. Front-end stages as implemented in the prototype; back-end optimisation and loop closure as provided by cuVSLAM / RTAB-Map.

14.1 Stereo depth

A stereo camera with baseline B and focal length f (pixels) measures depth from disparity d ; a disparity error σ_d grows quadratically into a depth error:

$$z = \frac{fB}{d}, \quad \sigma_z = \frac{z^2}{fB} \sigma_d \quad (19)$$

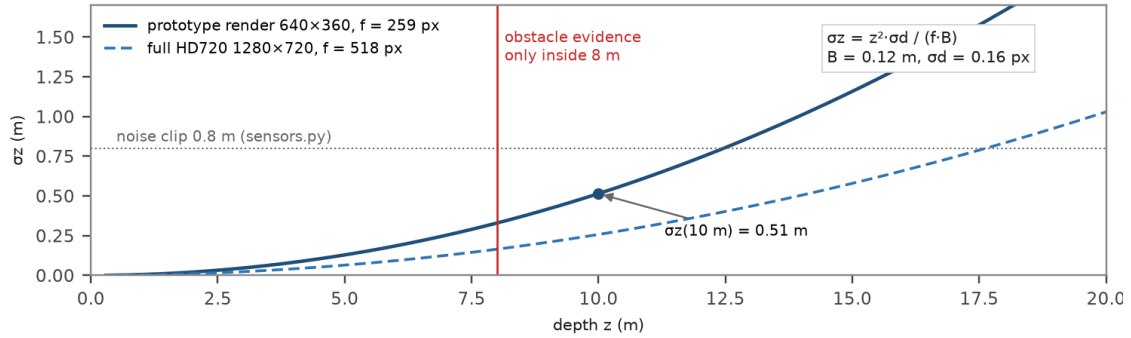


Figure 25. Depth uncertainty of a ZED 2i ($B = 0.12$ m) versus range — $\sigma_z = 0.51$ m at 10 m; the prototype only accepts obstacle evidence inside 8 m.

14.2 The prototype front-end IMPLEMENTED · MuJoCo prototype

The MuJoCo prototype runs a real stereo-depth VO front-end on the rendered ZED 2i images (not a replay of ground truth):

- **Features.** ORB on the histogram-equalised left image: FAST corners (threshold 12) at 8 pyramid levels (scale 1.2), up to 1500 features, each with a 256-bit rotated-BRIEF descriptor (31 px patch).
- **3-D landmarks.** Each corner with valid depth (0.3–15 m) is back-projected with the pinhole model: $X = (u - c_x) z / f_x$, $Y = (v - c_y) z / f_y$, $Z = z$, in the keyframe's optical frame.
- **Matching.** Brute-force Hamming distance, two nearest neighbours, Lowe ratio test $d_1 < 0.78 d_2$; at least 14 matches.
- **Pose.** PnP-RANSAC on the 3-D $\leftrightarrow$ 2-D pairs minimises the reprojection error $\sum \|u_i - \pi(K(R X_i + t))\|^2$ with a 2.5 px inlier threshold, 220 iterations, confidence 0.995, then Levenberg–Marquardt refinement on the inliers (≥ 10). RANSAC needs $N = \log(1 - p) / \log(1 - w^s)$ iterations for inlier ratio w and sample size s .
- **Chaining.** $T_{\text{world, cam}} = T_{\text{world, kf}} \cdot (T_{\text{cur, kf}})^{-1}$; a new keyframe is taken after 0.45 m, 11° or fewer than 40 inliers; 25 % of each keyframe's landmarks join a sparse map (≤ 9000 points).

It has no loop closure or bundle adjustment, so drift grows monotonically: 0.98 % of path in the committed run and up to 4.3 % in later runs (PnP-RANSAC is not bit-exact between runs), with 0 tracking losses over ≈ 260 keyframes.

14.3 Production engines EXPERIMENTAL / NOT WIRED

cuVSLAM (Isaac ROS Visual SLAM, installed at version 15.0.0 with Isaac ROS 4.6, CUDA 13.2) is NVIDIA's GPU-accelerated stereo/RGB-D visual-inertial odometry with sparse bundle adjustment and loop closure. Integration findings from this workspace: it must be started from a launch file (the bare binary fails on its Bazel runfiles), and single-camera RGB-D mode needs all of `tracking_mode: 2, num_cameras: 1, min_num_images: 1, depth_camera_id: 0` — with the default two cameras it silently publishes nothing. Its odometry will feed the EKF's `odom0` input with TF publishing disabled (rule 3).

RTAB-Map was the Phase 2 choice for the Gazebo stack: graph-based SLAM with appearance-based loop closure (bag-of-words over visual words) and working/long-term memory management that bounds computation on long missions. Rule 4 applies: its odometry comes in on a topic, never through `odom_frame_id`. **slam_toolbox** (lidar SLAM: correlative scan matching, Ceres pose-graph optimisation, 0.10 m resolution, loop search within 8 m) is wired behind `slam:=true` as an owner for `map→odom`; the `sync` node is used because the `async` node has an activation race under simulation time. On this 3-D terrain a 2-D scan disagrees with odometry geometry on slopes, and its corrections degraded the pose after ~50 m — hence the visual route for `map→odom`.

15 QSLAM innovations — designed modules

QSLAM DESIGN · planned The registered QSLAM idea adds three modules on top of the proven components above. They are specified here so the design is complete; none is in the repository yet (Figure 26).

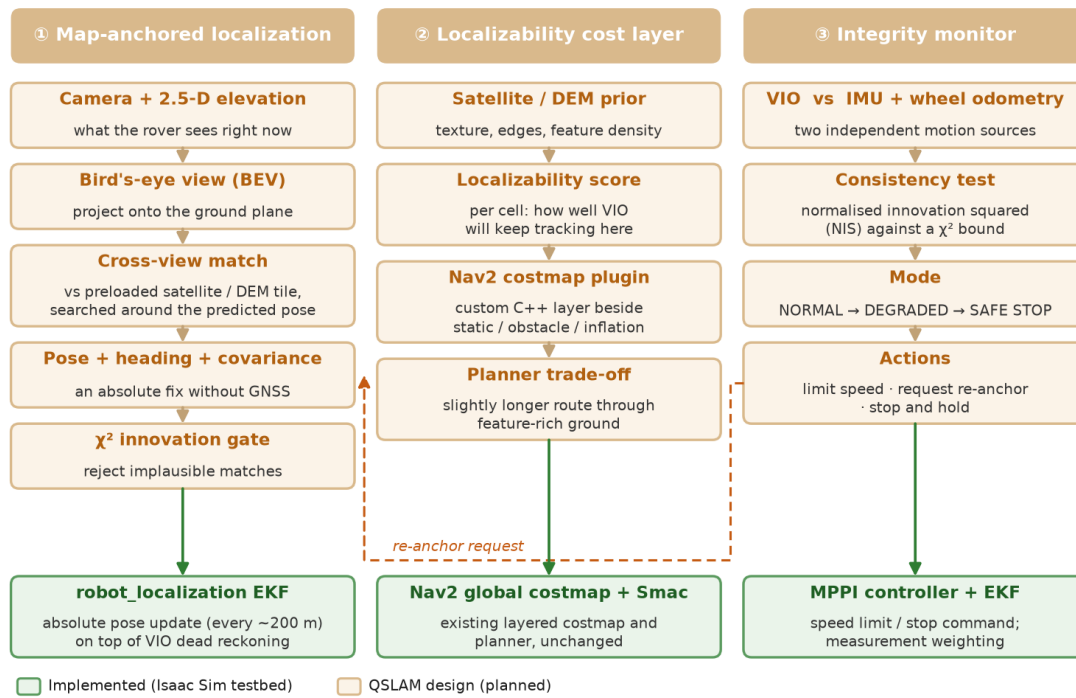


Figure 26. The three QSLAM innovations and where they plug into the implemented EKF and Nav2 stack.

15.1 Map-anchored global localization

This module plays the role GPS used to play, passively. (1) A **bird's-eye view** (BEV) of the surroundings is built by projecting camera pixels (or their semantic classes) onto the ground using depth and the GPU 2.5-D elevation map. (2) A tile of the **preloaded orthoimage and DEM** is cropped around the EKF's predicted pose, sized by its covariance. (3) A cross-view matcher (learned embeddings in the style of Shi & Li 2022 and OrienterNet 2023, or normalised cross-correlation on elevation shape) scores every candidate offset ($\Delta x, \Delta y, \Delta \psi$); the peak gives a UTM pose and heading and its sharpness a covariance. (4) The match is accepted only if it is statistically consistent with the filter:

$$d^2 = \boldsymbol{\nu}^T \mathbf{S}^{-1} \boldsymbol{\nu} < \chi_{3, 0.99}^2 = 11.34 \quad (20)$$

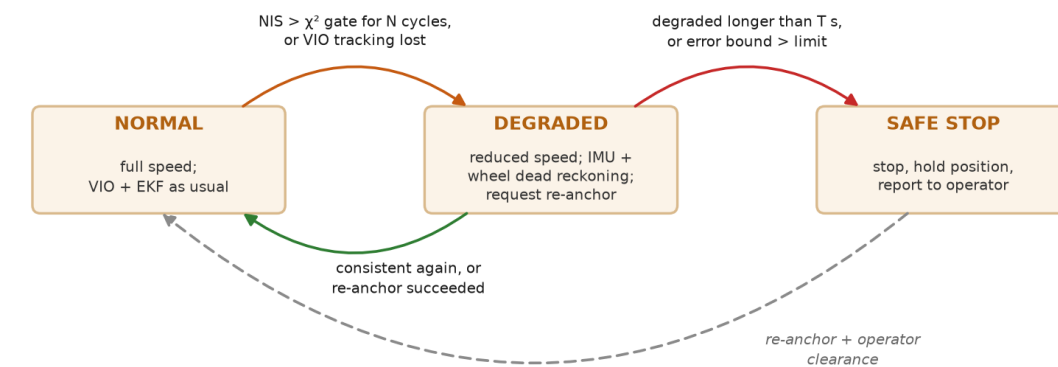
(5) An accepted match is fused as an absolute (x, y, ψ) update and takes ownership of `map→odom`, re-anchoring every ~200 m. The same matcher also provides the initial pose and heading, removing the need for a surveyed start point and a compass. The team's proof-of-concept study (reported in the idea deck; not reproduced in this repository) cut 3 km GNSS-free position error from 89.6 m to 10.6 m (–88 %).

15.2 Localizability-aware navigation

Visual odometry fails on featureless ground (sand, water, uniform grass — cf. the low-texture patch in §6). A custom Nav2 costmap layer (C++ plugin) scores each cell by predicted localizability $L \in [0, 1]$ from orthoimage texture/feature density and from feature counts observed online, adding $c_{loc} = w \cdot (1 - L)$ to the traversability cost. The planner then accepts a slightly longer route that keeps the camera tracking. PoC (idea deck): -59 % arrival error for +18 % path length.

15.3 Localization integrity monitor

Every cycle the monitor cross-checks visual odometry against the IMU and wheel odometry: normalised innovation squared (NIS) against a χ^2 gate, VIO tracking status and feature count. It drives a three-state supervisor (Figure 27): NORMAL; DEGRADED — slow down, prefer high-localizability cells, request a re-anchor, bridge with IMU + wheels; SAFE-STOP — stop and hold when the error bound exceeds the mission limit. The system degrades visibly instead of drifting silently.



QSLAM design (Phase 4) — thresholds N , T and limits are to be tuned; this monitor is not in the repository yet.

Figure 27. Integrity-monitor state machine (QSLAM design).

15.4 Perception and safety layers

Semantic segmentation (SegFormer fine-tuned on the RELLIS-3D and RUGD off-road datasets, TensorRT-optimised) labels drivable ground, vegetation, water and obstacles; elevation_mapping_cupy fuses depth into a GPU 2.5-D elevation map from which slope, step height and roughness become traversability costs, fused with the semantic classes. Nav2's Collision Monitor adds a last-line safety check: polygon zones around the footprint that slow or stop the vehicle independently of the planner. Target: ≥ 15 FPS perception on a Jetson AGX Orin.

16 Simulation platform, vehicle and results

16.1 Isaac Sim testbed IMPLEMENTED · Isaac testbed

Isaac Sim 6.0.1 loads the USD stage written by build_scene.py: the terrain mesh from the DEM with its texture, the roofed tunnel, obstacles, trees, rocks, four humans and lighting. PhysX steps at 60 Hz and rendering at 30 Hz. An OmniGraph action graph (OnPlaybackTick → ROS2Context, ROS2PublishClock, two ROS2CameraHelpers for RGB and depth, ROS2CameraInfoHelper, ROS2RtxLidarHelper, ROS2PublishJointState, IsaacReadSimulationTime) publishes sensors; an in-process rclpy node handles /cmd_vel, odometry, IMU, ground truth and moves the patrolling humans along their waypoints on the DEM surface.

Skid-steer drive. A twist (v, ω) becomes wheel speeds $\omega_{L,R} = (v \mp \omega \cdot G \cdot T/2)/r$ with track $T = 0.70$ m, $r = 0.15$ m, limited to 30 rad/s. PhysX reproduces the real platform's scrub: with stock friction only ≈ 6 % of the commanded yaw rate was achieved. Wheel friction μ 0.45/0.40, drive damping 5000, max force 900 and a measured yaw gain $G = 6.5$ (spin ratio 0.398 at $G = 2.6$, calibrate_yaw.py) restore tracking (Figure 28).

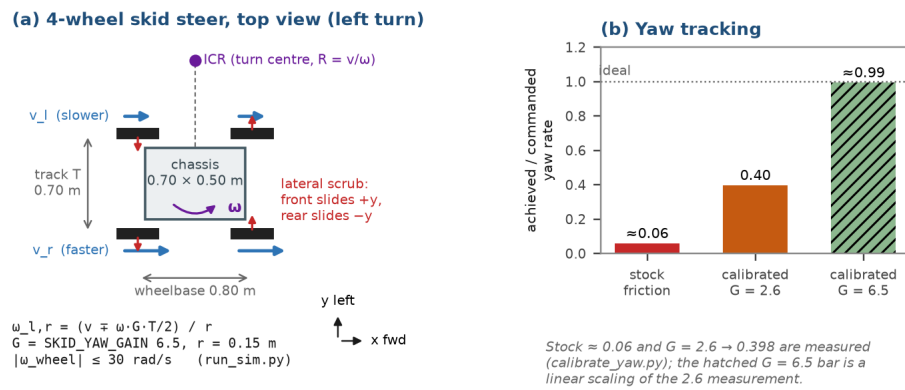


Figure 28. (a) Skid-steer geometry and the command mapping; (b) yaw-rate tracking before and after calibration.

16.2 Results

Table 11. Measured results. Isaac runs: seed 42, goal E 403 562.52 N 1 450 024.12. The committed run is the README's reference run (123.5 m = Nav2's initial plan length; straight line 119.4 m); run_final and run_1080p are the recorded runs, errors at the last logged sample.

Measure	Committed run	run_final	run_1080p	MuJoCo prototype
Goal reached	yes, via tunnel	yes, via tunnel	yes, via tunnel	yes
Route / distance travelled	123.5 m plan	189.0 m	295.1 m	121.5–123.8 m
Localization error at end	0.98 m	1.09 m	1.42 m	1.20–5.33 m (VO drift)
Arrival error (truth → goal)	0.23 m	1.33 m	1.20 m	1.91–5.68 m
Recoveries	1	7	1	—
Mission time (sim)	≈ 460 s	968 s	1417 s	127–131 s
Obstacle contacts	not logged	not logged	not logged	0 (21 obstacles)

Recording pipeline. record_mission.sh runs the headless simulator with three follow cameras to 1080p MP4, the navigation stack, an RViz screen capture, the telemetry logger (mission_logger.py, 5 Hz CSV) and the goal client, then renders a highlight reel (make_highlight.py) and a PDF mission report (make_mission_report.py).

17 Technology stack, limitations and roadmap

Table 12. Technologies implemented or specified in QSLAM.

Layer	Technologies	Role
Middleware	ROS 2 Jazzy (rclpy/rclcpp, DDS, TF2, lifecycle, actions)	Message passing, time, transforms
Navigation	Nav2: bt_navigator (BehaviorTree.CPP v4), Smac Planner 2D (Hybrid-A* planned), MPPI controller, costmap_2d, behaviours, map_server, lifecycle_manager	Planning, control, supervision
Localization	robot_localization EKF; Isaac ROS cuVSLAM; RTAB-Map; slam_toolbox (Karto + Ceres)	State estimation
Geospatial	GeoTIFF via rasterio/GDAL, pyproj (PROJ), EPSG:32644, QGIS-compatible outputs; Bhuvan/CartoDEM planned	Prior maps, UTM↔map
Vision	OpenCV (ORB, BFMatcher, solvePnP/Ransac, LM refine); SegFormer + TensorRT, elevation_mapping_cupy (planned)	Visual odometry, perception
Simulation	NVIDIA Isaac Sim 6.0.1 (USD, PhysX, RTX lidar, OmniGraph ROS 2 bridge); MuJoCo; Gazebo Harmonic + ros_gz	Validation
Compute	RTX 4090 + i9-13900K (testbed); Jetson AGX Orin (target)	Onboard execution
Tooling	Python 3.12, NumPy, Matplotlib, ReportLab, ffmpeg, RViz 2, colcon	Build, analysis, reporting

Limitations (stated, not hidden) and roadmap

- Testbed EKF inputs are idealised (no VIO/AHRS noise yet — Phase 2b); drift is bounded only by distance until map anchoring or loop closure (§15.1) is in the loop.
- The lidar is 2-D: negative obstacles come only from the prior DEM until the 2.5-D elevation map exists (Phase 3); slam_toolbox degraded on 3-D terrain and is not in the measured configuration. The roadmap and its targets are in Table 4.

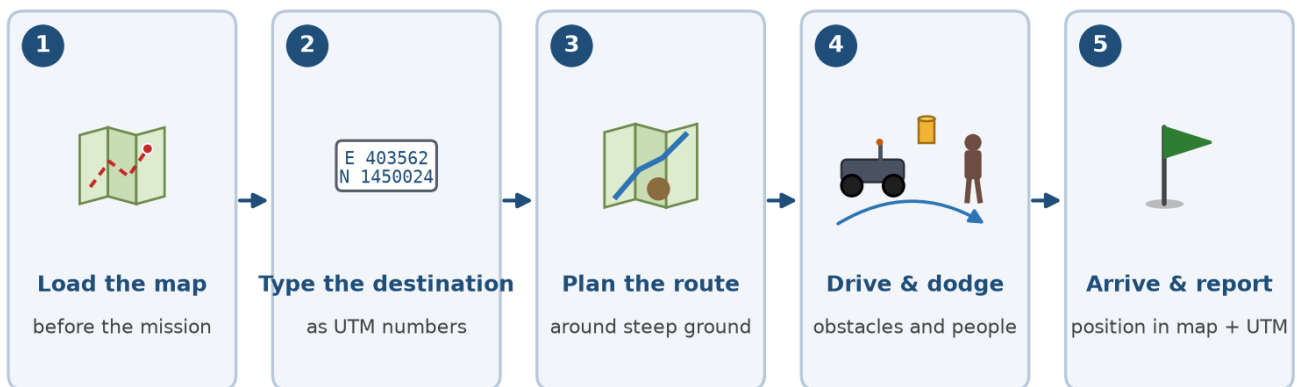
Part B — QSLAM explained for everyone

B1 What QSLAM is, in one page

Picture a small robotic vehicle, about the size of a large suitcase on four wheels. Someone gives it one instruction: “**go to this point on the map.**” Nobody drives it. On the way it has to cross open ground, climb a gentle hill, keep away from a mound that is too steep, find the one tunnel through an earth wall, and steer around barrels, logs, rocks and people walking across its path — and it must do all of this **without GPS**.

Why without GPS? The GPS signal is a very weak radio signal from satellites about 20,000 km away. It does not reach inside tunnels and mines, it fades under thick trees and between tall buildings, and in a conflict it can be jammed (drowned out) or spoofed (faked). Those are exactly the places where soldiers, rescuers and miners would most like to send a robot instead of a person.

QSLAM is our team's system for this problem (SIH 2026, problem statement SIH26126). “SLAM” stands for *Simultaneous Localization and Mapping*: a robot working out where it is while it makes sense of what it sees. QSLAM is software — the “brain” — and it has been built and tested in a realistic 3-D simulation of a 150 m × 150 m outdoor site.



Step 1 happens once, before the vehicle moves; steps 2–5 run on the vehicle, live, with no network connection.

Figure 29. A QSLAM mission in five steps.

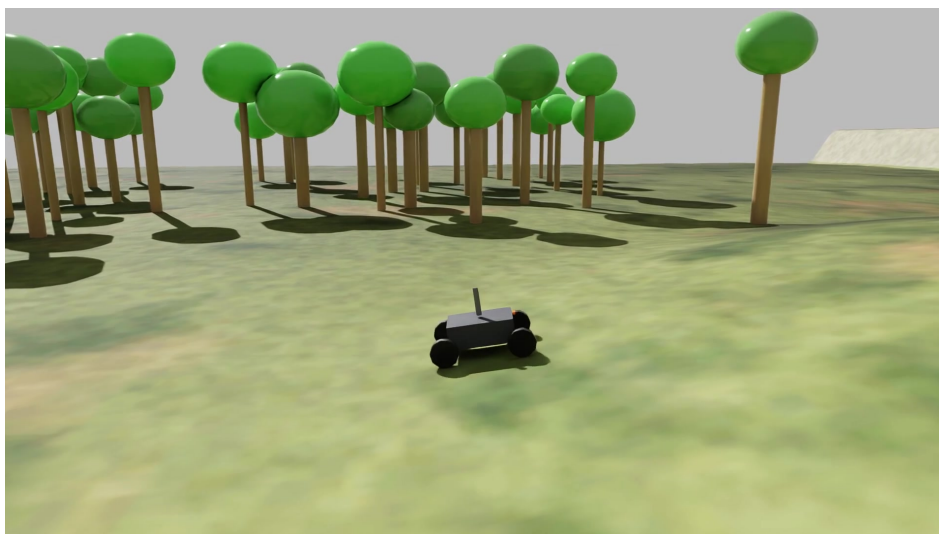


Figure 30. The simulated rover passing a grove of trees during a test mission (Isaac Sim).

B2 Does it use GPS? No. So how does it know where it is?

Straight answer: QSLAM does not use GPS at all. There is no GPS receiver in the system — not switched off, simply not there. So it cannot lose a GPS signal, and nobody can jam or fake one to mislead it.

It works out its position the way a careful hiker without a phone would, using four ideas:

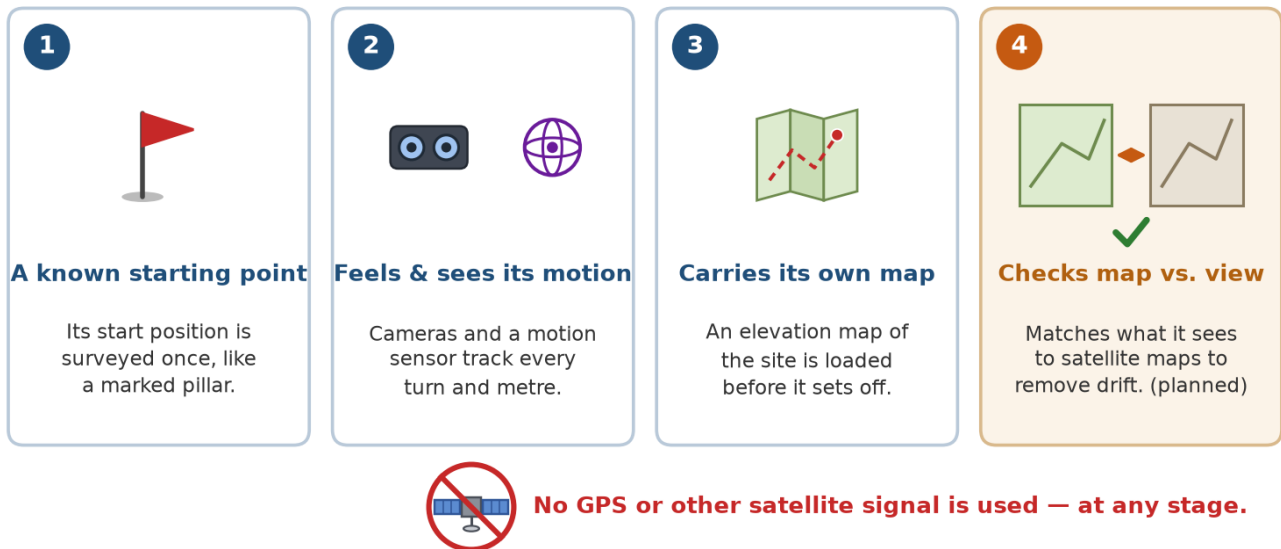


Figure 31. How QSLAM knows where it is without GPS.

- **1. It starts from a known spot.** The launch point was measured once, in advance — like starting a walk from your own front door, whose address you know.
- **2. It feels and watches its own movement.** The camera sees the ground slide past (like watching the road from a car window), an inertial sensor feels every turn (like your inner ear) and a heading sensor gives direction (like a compass). A mathematical “referee” called a *Kalman filter* combines them, trusting each source as much as it deserves, into one best guess of position — updated 30 times a second.
- **3. It carries its own map.** Before the mission it is given a height map of the area, so it already knows where the slopes, the ditch and the tunnel are, and plans a sensible route.
- **4. It recognises places (being added).** Small errors add up over distance. The planned “map anchoring” step compares the ground the camera sees, viewed from above, with a satellite picture of the area — like a hiker matching the shape of a hill to the map — and corrects the drift.

How good is it? In the simulated tests, after driving about 190 m entirely on its own, the robot's idea of its position was about 1 metre off — roughly the length of the vehicle plus a bit. (The sensors in the simulation are cleaner than real ones; real-world numbers will be measured next.)

How can it accept a UTM “GPS-style” coordinate without GPS? UTM is simply a grid that names every place on Earth in metres (an easting and a northing), like a giant sheet of graph paper. The site's reference point was surveyed once, so turning the destination into “118 m east and 18 m north of where you start” is a subtraction. No satellite is needed for arithmetic.

B3 Is QSLAM an online or an offline system?

Short answer: offline. QSLAM needs no internet, no cloud server, no mobile network and no satellite link. Every calculation happens on the vehicle's own computer. If all communication is cut, the mission simply continues.

People use the word “online” in two different ways, so here is each one answered separately:

Question	Answer for QSLAM
Is it connected to the internet or a server while it works?	No. Nothing is sent or received over a network during the mission. The maps it needs are copied onto the vehicle beforehand.
Does it work live, in real time?	Yes. It reads its sensors, updates its position and re-plans its steering many times per second (steering 15 times a second, position 30 times a second).
Does a person have to steer it?	No. A person only types the destination once. Everything after that is automatic.

What it needs

- ✓ An onboard computer (GPU)
- ✓ Cameras (stereo / RGB-D)
- ✓ A motion sensor (IMU)
- ✓ A laser scanner (lidar)
- ✓ Map files loaded before the mission
- ✓ One surveyed start point

What it does NOT need

- ✗ GPS / GNSS or any satellite signal
- ✗ Internet connection
- ✗ Cloud server
- ✗ Mobile / cellular network
- ✗ A remote driver with a joystick

Figure 32. What QSLAM needs, and what it does not need.

Before the mission – done once, offline	During the mission – on the vehicle, live
Load the height map of the area (a “GeoTIFF” file)	Read camera, laser scanner and motion sensors
Record the surveyed reference point of the site	Estimate its own position (Kalman filter)
Prepare the planner's map: mark slopes steeper than 15° as no-go	Plan the route and re-check it every second
(Planned) Load satellite image tiles of the area	Steer around obstacles and people, 15 times a second
	Report progress in map metres and in UTM

B4 One mission, step by step — and what has been proven

- **Preparation.** The site's height map and reference point are loaded. The planner marks everything steeper than 15° as no-go.
- **Destination.** The operator types one line: easting 403 562.52, northing 1 450 024.12. QSLAM turns that into “118 m east, 18 m north of the start”.
- **Plan.** The route planner searches the map for the cheapest safe path. The earth wall is too steep everywhere except at the tunnel, so the route goes through the tunnel.
- **Drive.** A laser scanner spots things the map never contained — barrels, logs, rocks, walking people. The steering controller imagines 1800 possible manoeuvres for the next few seconds, fifteen times every second, and picks the safest one that still makes progress. If it gets stuck, it backs up, turns or waits, then tries again.
- **Arrive.** It stops within about a metre of the destination and reports success.

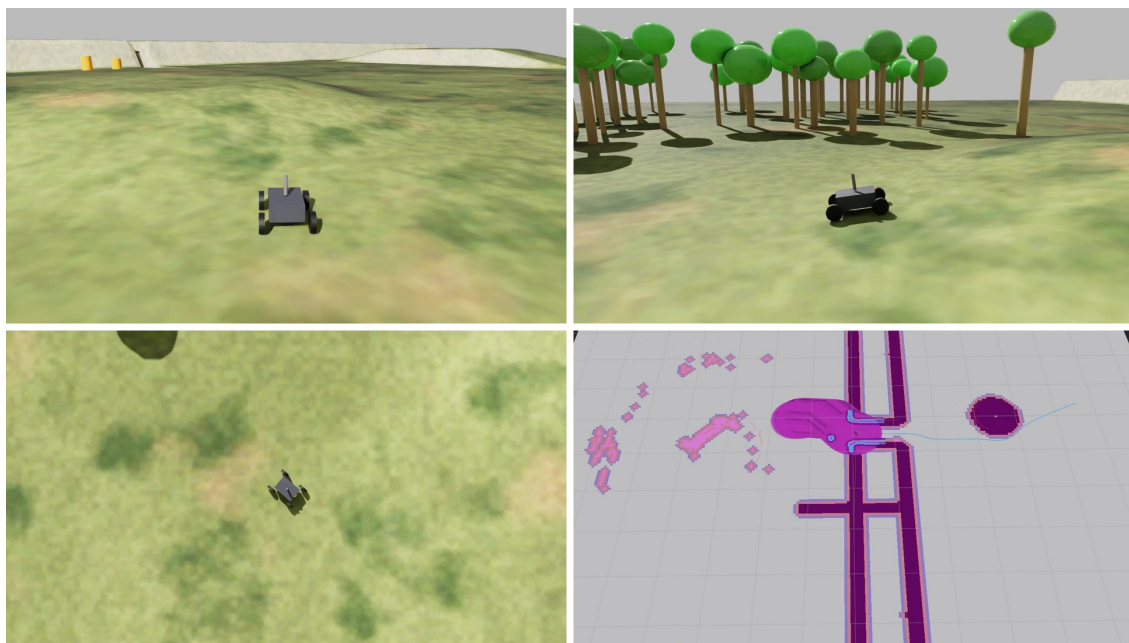


Figure 33. A recorded test mission from four viewpoints; bottom right is the robot's own view of its maps and planned route.

What has been shown so far (in simulation)

Question	Result
Did it reach the destination?	Yes — in every recorded test, going through the tunnel.
How far off was its own position estimate?	About 1 to 1.4 m after 123–295 m of driving, with no GPS.
Did it hit anything or anyone?	In the prototype, which counts every contact: none, across 21 obstacles. The Isaac test world does not log contacts yet.
Did it ever use GPS?	Never — there is no GPS in the software.

Not yet proven: everything so far is in simulation; the camera-only position tracking is being connected in place of the simulator's idealised motion data, and the satellite map-matching step is designed but not built. Real-vehicle trials come after that.

B5 Questions people usually ask

Could an enemy jam or fake its navigation?

There is no GPS signal to jam or fake. Its camera and motion sensors only receive, they do not transmit; the test vehicle's laser scanner is used only to see nearby obstacles.

What if the camera cannot see — dust, darkness, a blank wall?

The motion sensors carry it through short gaps. The planned integrity monitor notices when vision becomes unreliable, slows the vehicle down, and stops it safely if needed; a thermal camera is an optional upgrade for night.

How accurate does it need to be?

The design goal is to arrive within 2 % of the distance driven — within 2 m after 100 m.

What happens if the way is completely blocked?

It tries recovery moves and new routes. If no safe route exists, it stops and reports that the goal could not be reached, rather than taking a risk.

Can it work in a place it has never been?

Yes, if a height map of the area exists — for India these are available from national sources such as ISRO's Bhuvan portal — plus one known reference point. With map anchoring, even that point comes from the satellite image.

What computer does it need?

Tests run on a desktop graphics computer. The target is an NVIDIA Jetson AGX Orin — a palm-sized computer designed for robots.

Mini glossary

Word	Plain meaning
GPS / GNSS	Satellite positioning. QSLAM does not use it.
UTM	A world-wide grid of coordinates in metres (easting, northing).
DEM / GeoTIFF	A height map of the ground, stored as an image file that knows where on Earth it is.
SLAM / VSLAM	Working out where you are while mapping what you see; “V” means using a camera.
Kalman filter (EKF)	The maths that blends several imperfect sensors into one best position estimate.
Lidar	A laser scanner that measures distances to nearby objects.
IMU	A motion sensor that feels turning and acceleration, like the inner ear.
ROS 2 / Nav2	Standard open-source robot software on which QSLAM is built.
Simulation	A physics-accurate virtual world used to test the robot safely and repeatably.